AI Open 3 (2022) 111–132

Contents lists available at ScienceDirect

AI Open

journal homepage: www.keaipublishing.com/en/journals/ai-open

A survey of transformers

Tianyang Lin, Yuxin Wang, Xiangyang Liu, Xipeng Qiu *

School of Computer Science, Fudan University, Shanghai, 200433, China
Shanghai Key Laboratory of Intelligent Information Processing, Shanghai, 200433, China

ARTICLE INFO

Keywords:
Transformer
Self-attention
Pre-trained models
Deep learning

ABSTRACT

Transformers have achieved great success in many artificial intelligence fields, such as natural language processing, computer vision, and audio processing. Therefore, it is natural to attract lots of interest from academic and industry researchers. Up to the present, a great variety of Transformer variants (a.k.a. X-formers) have been proposed, however, a systematic and comprehensive literature review on these Transformer variants is still missing. In this survey, we provide a comprehensive review of various X-formers. We first briefly introduce the vanilla Transformer and then propose a new taxonomy of X-formers. Next, we introduce the various X-formers from three perspectives: architectural modification, pre-training, and applications. Finally, we outline some potential directions for future research.

1. Introduction

Transformer (Vaswani et al., 2017) is a prominent deep learning model that has been widely adopted in various fields, such as natural language processing (NLP), computer vision (CV) and speech processing. Transformer was originally proposed as a sequence-to-sequence model (Sutskever et al., 2014) for machine translation. Later works show that Transformer-based pre-trained models (PTMs) (Qiu et al., 2020) can achieve *state-of-the-art* performances on various tasks. As a consequence, Transformer has become the go-to architecture in NLP, especially for PTMs. In addition to language related applications, Transformer has also been adopted in CV (Parmar et al., 2018; Carion et al., 2020; Dosovitskiy et al., 2020), audio processing (Dong et al., 2018; Gulati et al., 2020; Chen et al., 2021) and even other disciplines, such as chemistry (Schwaller et al., 2019) and life sciences (Rives et al., 2021).

Due to the success, a variety of Transformer variants (a.k.a. X-formers) have been proposed over the past few years. These X-formers improve the vanilla Transformer from different perspectives.

1. *Model Efficiency*. A key challenge of applying Transformer is its inefficiency at processing long sequences mainly due to the computation and memory complexity of the self-attention module. The improvement methods include lightweight attention (e.g. sparse attention variants) and Divide-and-conquer methods (e.g., recurrent and hierarchical mechanism).
2. *Model Generalization*. Since the transformer is a flexible architecture and makes few assumptions on the structural bias of input data, it is hard to train on small-scale data. The improvement methods include introducing structural bias or regularization, pre-training on large-scale unlabeled data, etc.
3. *Model Adaptation*. This line of work aims to adapt the Transformer to specific downstream tasks and applications.

In this survey, we aim to provide a comprehensive review of the Transformer and its variants. Although we can organize X-formers on the basis of the perspectives mentioned above, many existing X-formers may address one or several issues. For example, sparse attention variants not only reduce the computational complexity but also introduce structural prior on input data to alleviate the overfitting problem on small datasets. Therefore, it is more methodical to categorize the various existing X-formers and propose a new taxonomy mainly according to their ways to improve the vanilla Transformer: architecture modification, pre-training, and applications. Considering the audience of this survey may be from different domains, we mainly focus on the general architecture variants and just briefly discuss the specific variants on pre-training and applications.

The rest of the survey is organized as follows. Section 2 introduces the architecture and the key components of Transformer. Section 3 clarifies the categorization of Transformer variants. Section 4~5 review the module-level modifications, including attention module, position encoding, layer normalization and feed-forward layer. Section 6 reviews the architecture-level variants. Section 7 introduces some of the representative Transformer-based PTMs. Section 8 introduces the application of Transformer to various different fields. Section 9 discusses some aspects of Transformer that researchers might find intriguing and summarizes the paper.

* Corresponding author at: School of Computer Science, Fudan University, Shanghai, 200433, China
E-mail address: xpqiu@fudan.edu.cn (X. Qiu).

https://doi.org/10.1016/j.aiopen.2022.10.001
Received 27 September 2022; Accepted 11 October 2022
Available online 20 October 2022

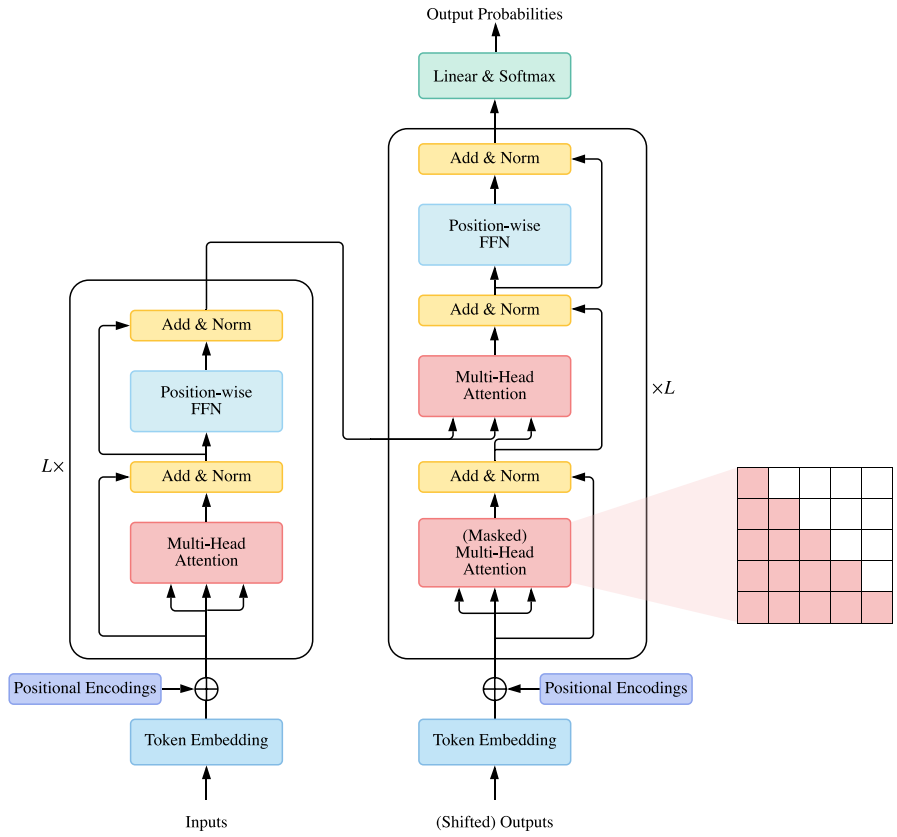


Fig. 1. Overview of vanilla Transformer architecture.

2. Background

2.1. Vanilla Transformer

The vanilla Transformer (Vaswani et al., 2017) is a sequence-to-sequence model and consists of an encoder and a decoder, each of which is a stack of L identical blocks. Each *encoder block* is mainly composed of a multi-head self-attention module and a position-wise feed-forward network (FFN). For building a deeper model, a residual connection (He et al., 2016) is employed around each module, followed by Layer Normalization (Ba et al., 2016) module. Compared to the encoder blocks, decoder blocks additionally insert cross-attention modules between the multi-head self-attention modules and the position-wise FFNs. Furthermore, the self-attention modules in the decoder are adapted to prevent each position from attending to subsequent positions. The overall architecture of the vanilla Transformer is shown in Fig. 1.

In the following subsection, we shall introduce the key modules of the vanilla Transformer.

2.1.1. Attention modules

Transformer adopts attention mechanism with Query–Key–Value (QKV) model. Given the packed matrix representations of queries $\mathbf{Q} \in \mathbb{R}^{N\times D_k}$, keys $\mathbf{K} \in \mathbb{R}^{M\times D_k}$, and values $\mathbf{V} \in \mathbb{R}^{M\times D_v}$, the scaled dot-product attention used by Transformer is given by[1]

$$\text{Attention}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{D_k}}\right)\mathbf{V} = \mathbf{A}\mathbf{V}, \tag{1}$$

where N and M denote the lengths of queries and keys (or values); D_k and D_v denote the dimensions of keys (or queries) and values; $\mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{D_k}}\right)$ is often called *attention matrix*; softmax is applied in a row-wise manner. The dot-products of queries and keys are divided by $\sqrt{D_k}$ to alleviate gradient vanishing problem of the softmax function.

Instead of simply applying a single attention function, Transformer uses multi-head attention, where the D_m-dimensional original queries, keys and values are projected into D_k, D_k and D_v dimensions, respectively, with H different sets of learned projections. For each of the projected queries, keys and values, and output is computed with attention according to Eq. (1). The model then concatenates all the outputs and projects them back to a D_m-dimensional representation.

$$\text{MultiHeadAttn}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \text{Concat}(\text{head}_1,\dots,\text{head}_H)\mathbf{W}^O, \tag{2}$$

$$\text{where head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V). \tag{3}$$

In Transformer, there are three types of attention in terms of the source of queries and key–value pairs:

- *Self-attention*. In Transformer encoder, we set $\mathbf{Q} = \mathbf{K} = \mathbf{V} = \mathbf{X}$ in Eq. (2), where $\mathbf{X}$ is the outputs of the previous layer.
- *Masked Self-attention*. In the Transformer decoder, the self-attention is restricted such that queries at each position can only attend to all key–value pairs up to and including that position. To enable parallel training, this is typically done by applying a mask function to the unnormalized attention matrix $\hat{\mathbf{A}} = \exp(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{D_k}})$, where the illegal positions are masked out by setting $\hat{A}_{ij} = -\infty$ if $i < j$. This kind of self-attention is often referred to as autoregressive or causal attention.[2]

[1] If not stated otherwise, we use row-major notations throughout this survey (e.g., the ith row in $\mathbf{Q}$ is the query $\mathbf{q}_i$) and all the vectors are row vectors by default.

[2] This term seems to be borrowed from the *causal system*, where the output depends on past and current inputs but not future inputs.

- *Cross-attention*. The queries are projected from the outputs of the previous (decoder) layer, whereas the keys and values are projected using the outputs of the encoder.

2.1.2. Position-wise FFN

The position-wise FFN[3] is a fully connected feed-forward module that operates separately and identically on each position

$$\mathrm{FFN}(\mathbf{H}') = \mathrm{ReLU}(\mathbf{H}'\mathbf{W}^1 + \mathbf{b}^1)\mathbf{W}^2 + \mathbf{b}^2, \tag{4}$$

where $\mathbf{H}'$ is the outputs of previous layer, and $\mathbf{W}^1 \in \mathbb{R}^{D_m \times D_f}, \mathbf{W}^2 \in \mathbb{R}^{D_f \times D_m}, \mathbf{b}^1 \in \mathbb{R}^{D_f}, \mathbf{b}^2 \in \mathbb{R}^{D_m}$ are trainable parameters. Typically the intermediate dimension D_f of the FFN is set to be larger than D_m.

2.1.3. Residual connection and normalization

In order to build a deep model, Transformer employs a residual connection (He et al., 2016) around each module, followed by Layer Normalization (Ba et al., 2016). For instance, each Transformer encoder block may be written as

$$\mathbf{H}' = \mathrm{LayerNorm}(\mathrm{SelfAttention}(\mathbf{X}) + \mathbf{X}) \tag{5}$$

$$\mathbf{H} = \mathrm{LayerNorm}(\mathrm{FFN}(\mathbf{H}') + \mathbf{H}'), \tag{6}$$

where $\mathrm{SelfAttention}(\cdot)$ denotes self attention module and $\mathrm{LayerNorm}(\cdot)$ denotes the layer normalization operation.

2.1.4. Position encodings

Since Transformer does not introduce recurrence or convolution, it is ignorant of positional information (especially for the encoder). Thus additional positional representation (Detailed discussion in Section 5.1) is needed to model the ordering of tokens.

2.2. Model usage

Generally, the Transformer architecture can be used in three different ways:

- *Encoder–Decoder*. The full Transformer architecture as introduced in Section 2.1 is used. This is typically used in sequence-to-sequence modeling (e.g., neural machine translation).
- *Encoder only*. Only the encoder is used and the outputs of the encoder are utilized as a representation for the input sequence. This is often used for Natural Language Understanding (NLU) tasks (e.g., text classification and sequence labeling).
- *Decoder only*. Only the decoder is used, where the encoder-decoder cross-attention module is also removed. This is typically used for sequence generation (e.g., language modeling).

2.3. Model analysis

To illustrate the computation time and parameter requirements of the Transformer, we analyze the two core components of the Transformer (i.e., the self-attention module and the position-wise FFN) in Table 1. We assume that the hidden dimension D_m of the model is D, and that the input sequence length is T. The intermediate dimension of FFN is set to $4D$ and the dimension of keys and values are set to D/H as in Vaswani et al. (2017).

When the input sequences are short, the hidden dimension D dominates the complexity of self-attention and position-wise FFN. The bottleneck of Transformer thus lies in FFN. However, as the input sequences grow longer, the sequence length T gradually dominates the complexity of these modules, in which case self-attention becomes the bottleneck of Transformer. Furthermore, the computation of self-attention requires that a $T \times T$ attention distribution matrix is stored, which makes the computation of Transformer infeasible for long-sequence scenarios (e.g., long text documents and pixel-level modeling of high-resolution images). One shall see that the goal of increasing the efficiency of Transformer generally leads to the long-sequence compatibility of self-attention, as well as the computation and parameter efficiency of position-wise FFN for ordinary settings.

Table 1
Complexity and parameter counts of self-attention and position-wise FFN.

Module	Complexity	#Parameters
self-attention	$\mathcal{O}(T^2 \cdot D)$	$4D^2$
position-wise FFN	$\mathcal{O}(T \cdot D^2)$	$8D^2$

Table 2
Per-layer complexity, minimum number of sequential operations and maximum path lengths for different layer types. T is the sequence length, D is the representation dimension and K is the kernel size of convolutions (Vaswani et al., 2017).

Layer type	Complexity per layer	Sequential operations	Maximum path length
Self-Attention	$\mathcal{O}(T^2 \cdot D)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Fully Connected	$\mathcal{O}(T^2 \cdot D^2)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Convolutional	$\mathcal{O}(K \cdot T \cdot D^2)$	$\mathcal{O}(1)$	$\mathcal{O}(\log_K(T))$
Recurrent	$\mathcal{O}(T \cdot D^2)$	$\mathcal{O}(T)$	$\mathcal{O}(T)$

2.4. Comparing Transformer to other network types

2.4.1. Analysis of self-attention

As a central piece of Transformer, self-attention comes with a flexible mechanism to deal with variable-length inputs. It can be understood as a fully connected layer where the weights are dynamically generated from pairwise relations from inputs. Table 2 compares the complexity, sequential operations, and maximum path length[4] of self-attention with three commonly used layer types. We summarize the advantages of self-attention as follows:

1. It has the same maximum path length as fully connected layers, making it suitable for long-range dependencies modeling. Compared to fully connected layers, it is more parameter-efficient and more flexible in handling variable-length inputs.
2. Due to the limited receptive field of convolutional layers, one typically needs to stack a deep network to have a global receptive field. On the other hand, the constant maximum path length enables self-attention to model long-range dependencies with a constant number of layers.
3. The constant sequential operations and maximum path length make self-attention more parallelizable and better at long-range modeling than recurrent layers.

2.4.2. In terms of inductive bias

Transformer is often compared against convolutional and recurrent networks. Convolutional networks are known to impose the inductive biases of translation invariance and locality with shared local kernel functions. Similarly, recurrent networks carry the inductive biases of temporal invariance and locality via their Markovian structure (Battaglia et al., 2018). On the other hand, the Transformer architecture makes few assumptions about structural information of data. This makes Transformer a universal and flexible architecture. As a side effect, the lack of structural bias makes Transformer prone to overfitting for small-scale data.

[3] The parameters are shared across different positions, thus the position-wise FFN can also be understood as two convolution layers with kernel size of 1.

[4] The maximum length of the paths forward and backward signals have to traverse to get from any input position to arbitrary output position. Shorter length implies a better potential for learning long-range dependencies.

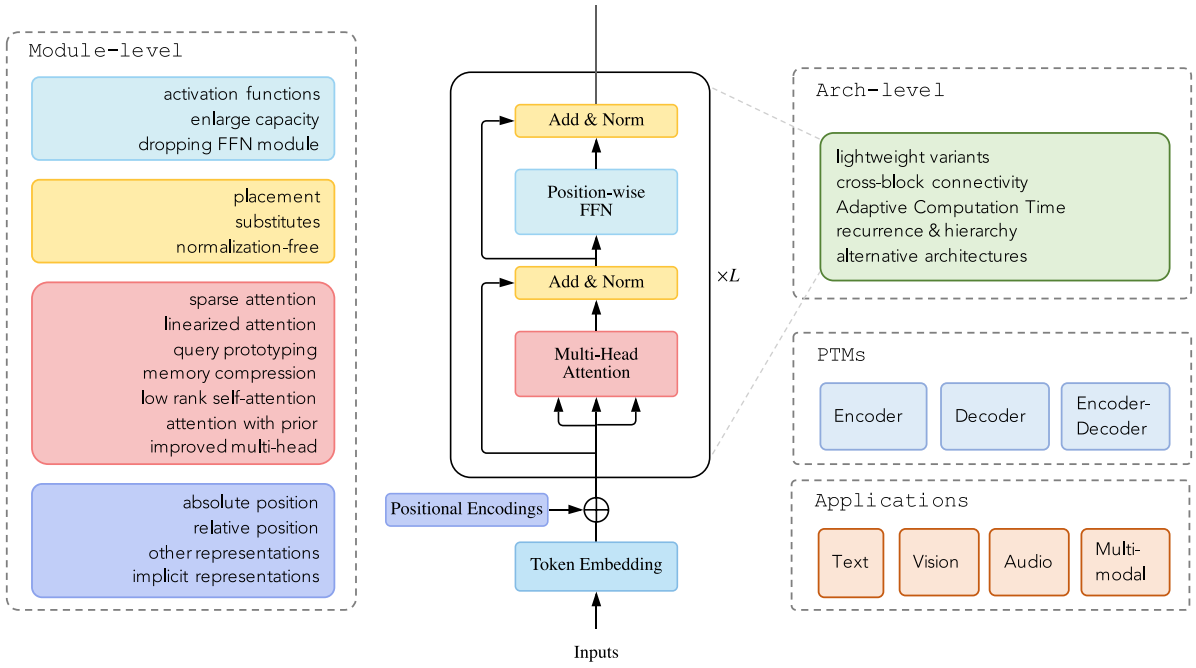


Fig. 2. Categorization of Transformer variants.

Another closely related network type is Graph Neural Networks (GNNs) with message passing (Wu et al., 2021a). Transformer can be viewed as a GNN defined over a complete directed graph (with self-loop) where each input is a node in the graph. The key difference between Transformer and GNNs is that Transformer introduces no prior knowledge over how input data are structured — the message passing process in Transformer solely depends on similarity measures over the content.

3. Taxonomy of Transformers

A wide variety of models have been proposed so far based on the vanilla Transformer from three perspectives: types of architecture modification, pre-training methods, and applications. Fig. 2 gives an illustrations of our categorization of Transformer variants.

Fig. 3 illustrates our taxonomy and some representative models.

In this survey, we focus on reviewing the works on architecture modifications. Since the attention module is the key component of Transformer, we solely describe the attention-related variants in Section 4 and introduce the other module-level variants in Section 5. Then Section 6 describes the other architecture-level variants. Finally, we briefly review the works on pre-training in Section 7 and applications in Section 8. There are some comprehensive surveys on the latter two categories of work, such as pre-trained models (PTMs) (Qiu et al., 2020) and visual Transformers (Han et al., 2021a; Khan et al., 2021).

4. Attention

Self-attention plays an important role in Transformer, but there are two challenges in practical applications.

1. *Complexity*. As discussion in Section 2.3, the complexity of self-attention is $\mathcal{O}(T^2 \cdot D)$. Therefore, the attention module becomes a bottleneck when dealing with long sequences.
2. *Structural prior*. Self-attention does no assume any structural bias over inputs. Even the order information is also needed to be learned from training data. Therefore, Transformer (w/o pre-training) is usually easy to overfit on small or moderate-size data.

The improvements on attention mechanism can be divided into several directions:

1. *Sparse Attention*. This line of work introduces sparsity bias into the attention mechanism, leading to reduced complexity.
2. *Linearized Attention*. This line of work disentangles the attention matrix with kernel feature maps. The attention is then computed in reversed order to achieve linear complexity.
3. *Prototype and Memory Compression*. This class of methods reduces the number of queries or key–value memory pairs to reduce the size of the attention matrix.
4. *Low-rank Self-Attention*. This line of work capture the low-rank property of self-attention.
5. *Attention with Prior*. The line of research explores supplementing or substituting standard attention with prior attention distributions.
6. *Improved Multi-Head Mechanism*. The line of studies explores different alternative multi-head mechanisms.

We will describe these attention variants at length in the rest of this section.

4.1. Sparse attention

In the standard self-attention mechanism, every token needs to attend to all other tokens. However, it is observed that for the trained Transformers the learned attention matrix $\mathbf{A}$ is often very sparse across most data points (Child et al., 2019). Therefore, it is possible to reduce computation complexity by incorporating structural bias to limit the number of query–key pairs that each query attends to. Under this limitation, we just compute the similarity score of the query–key pairs according to pre-defined patterns

$$\hat{\mathbf{A}}_{ij} = \begin{cases} \mathbf{q}_i \mathbf{k}_j^\top & \text{if token } i \text{ attends to token } j, \\ -\infty & \text{if token } i \text{ does not attend to token } j, \end{cases} \tag{7}$$

where $\hat{\mathbf{A}}$ is un-normalized attention matrix. In implementation the $-\infty$ item is usually not stored in memory so as to decrease memory footprint.

From another perspective, the standard attention can be regarded as a complete bipartite graph where each query receives information from

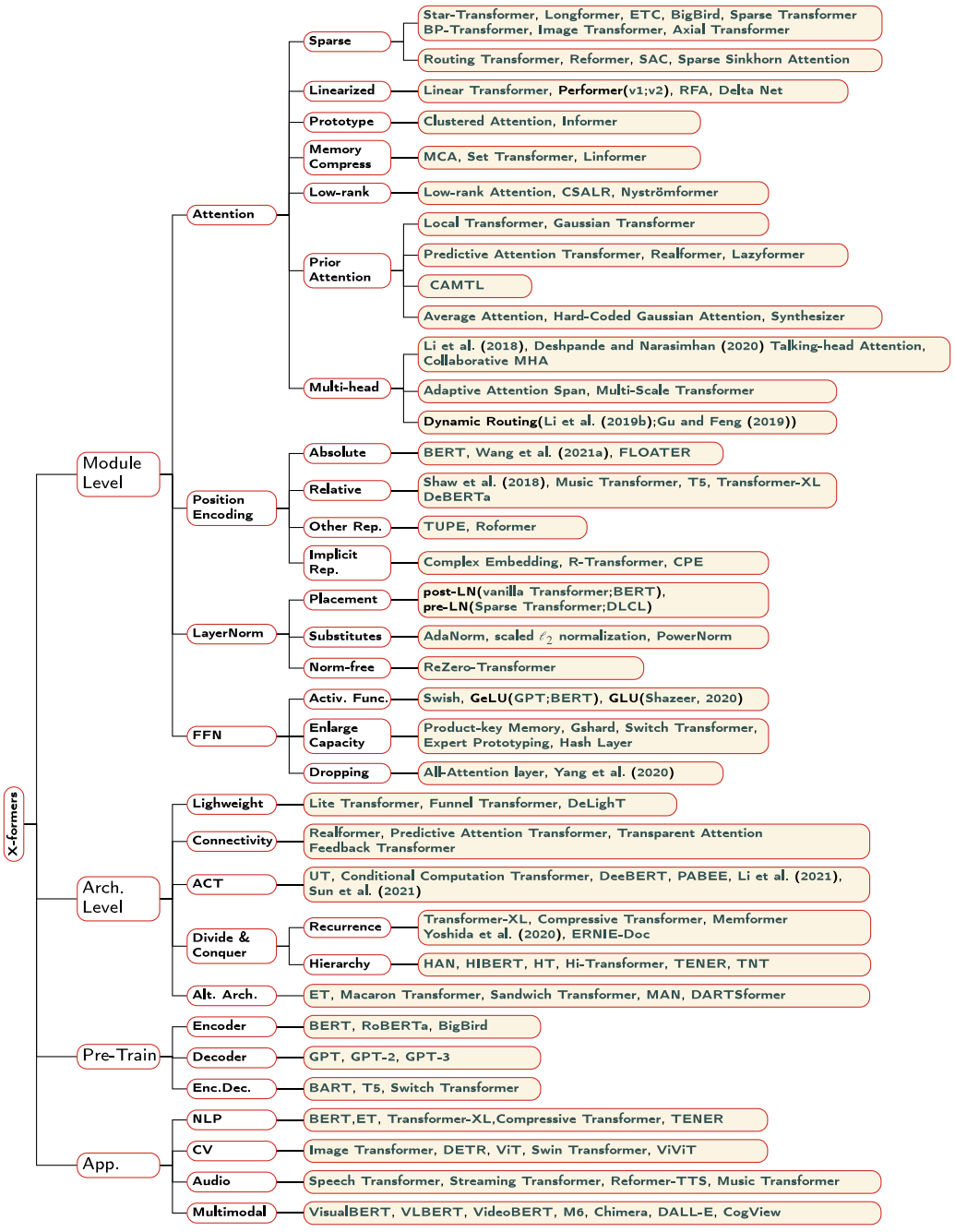


Fig. 3. Taxonomy of Transformers.

all memory nodes and updates its representation. The sparse attention can be considered as a sparse graph where some of the connections between nodes are removed.

Based on the metrics of determining the sparse connection, we categorize these approaches into two classes: *position-based* and *content-based* sparse attention.

4.1.1. Position-based sparse attention

In position-based sparse attention, the attention matrix is limited according to some pre-defined patterns. Although these sparse patterns vary in different forms, we find that some of them can be decomposed into some atomic sparse patterns.

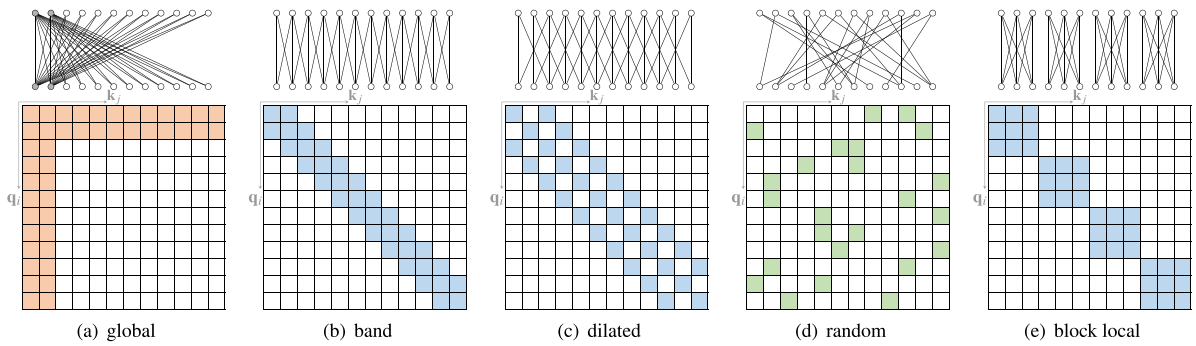
We first identify some atomic sparse patterns and then describe how these patterns are composed in some existing work. Finally, we introduce some extended sparse patterns for specific data types.

4.1.1.1. Atomic sparse attention.

There are mainly five types of atomic sparse attention patterns, as shown in Fig. 4.

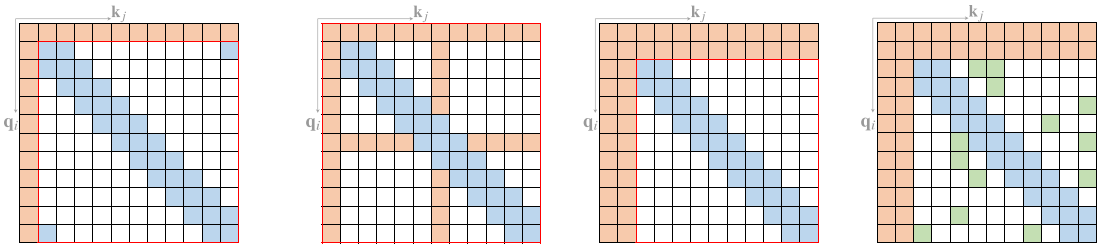
1. *Global Attention*. To alleviate the degradation of the ability to model the long-range dependencies in sparse attention, one can add some global nodes[5] as the hub for information propagation between nodes. These global nodes can attend all nodes in the sequence and the whole sequence attend to these global nodes, as illustrated in Fig. 4(a).
2. *Band Attention*(a.k.a *sliding window attention* or *local attention*). Since most data come with a strong property of locality, it is

[5] In practice, these global nodes can be selected from the sequence (internal global nodes) or virtual nodes with trainable parameters (external global nodes).



(a) global (b) band (c) dilated (d) random (e) block local

Fig. 4. Some representative atomic sparse attention patterns. The colored squares means corresponding attention scores are calculated and a blank square means the attention score is discarded.



(a) Star-Transformer (b) Longformer (c) ETC (d) BigBird

Fig. 5. Some representative compound sparse attention patterns. The red boxes indicate sequence boundaries.

natural to restrict each query to attend to its neighbor nodes. A widely adopted class of such sparse pattern is band attention, in which the attention matrix is a band matrix as illustrated in Fig. 4(b).

3. *Dilated Attention*. Analogous to dilated CNNs (van den Oord et al., 2016), one can potentially increase the receptive field of the band attention without increasing computation complexity by using a dilated window with gaps of dilation $w_d \geq 1$, as depicted in Fig. 4(c). This can be easily extended to *strided attention*, where the window size is not limited but the dilation w_d is set to a large value.
4. *Random Attention*. To increase the ability of non-local interactions, a few edges are randomly sampled for each query, as illustrated in Fig. 4(d). This is based on the observation that random graphs (e.g., Erdős–Rényi random graph) can have similar spectral properties with complete graphs that leads to a fast mixing time for random walking on graphs.
5. *Block Local Attention*. This class of attention segments input sequence into several non-overlapping query blocks, each of which is associated with a local memory block. All the queries in a query block attend to only the keys in the corresponding memory block. Fig. 4(e) depicts a commonly used case where the memory blocks are identical to their corresponding query blocks.

4.1.1.2. Compound sparse attention. Existing sparse attentions are often composed of more than one of the above atomic patterns. Fig. 5 illustrates some representative compound sparse attention patterns.

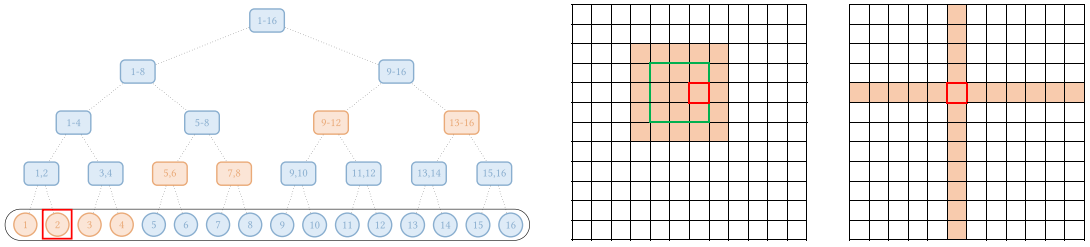
Star-Transformer (Guo et al., 2019a) uses a combination of band attention and global attention. Specifically, Star-Transformer just includes only a global node and a band attention with the width of 3, in which any pair of non-adjacent nodes are connected through a shared global node and adjacent nodes are connected directly with each other. This kind of sparse pattern forms a star-shaped graph among nodes. Longformer (Beltagy et al., 2020) uses a combination of band attention and internal global-node attention. The global nodes are chosen to be `[CLS]` token for classification and all question tokens for Question Answering tasks. They also replace some of the band attention heads in upper layers with dilated window attention to increase the receptive field without increasing computation. As a concurrent work to Longformer (Beltagy et al., 2020), Extended Transformer Construction (ETC) (Ainslie et al., 2020) utilizes combination of band attention and external global-node attention. ETC also includes a masking mechanism to handle structured inputs and adapt Contrastive Predictive Coding (CPC) (van den Oord et al., 2018) for pre-training. In addition to the band and global attention, BigBird (Zaheer et al., 2020) uses additional random attention to approximate full attention. Their theoretical analysis also reveals that the usage of a sparse encoder and sparse decoder can simulate any Turing Machine, which explains the success of those sparse attention models.

Sparse Transformer (Child et al., 2019) uses a factorized attention where different sparse patterns are designed for different types of data. For data with a periodic structure (e.g., images), it uses a composition of band attention and strided attention. Whereas for data without a periodic structure (e.g., text), it uses a composition of block local attention combined with global attention, where global nodes are from fixed positions in the input sequence.

4.1.1.3. Extended sparse attention. Apart from the above patterns, some existing studies have explored extended sparse patterns for specific data types.

For text data, BP-Transformer (Ye et al., 2019) constructs a binary tree where all tokens are leaf nodes and the internal nodes are span nodes containing many tokens. The edges in this graph are constructed so that each leaf node is connected to its neighbor leaf nodes and higher-level span nodes containing tokens from a longer distance. This approach can be seen as an extension of global attention, where global nodes are hierarchically organized and any pair of tokens are connected with paths in the binary tree. An abstract view of this method is illustrated in Fig. 6(a).

There are also some extensions for vision data. Image Transformer (Parmar et al., 2018) explores two types of attention: (1) flattening image pixels in raster-scan order and then applying block local sparse attention. (2) 2D block local attention, where query blocks and memory



(a) BPT (b) block local (2D) (c) axial (2D)

Fig. 6. Other types of sparse attentions. The red box indicates the query position, and the orange nodes/squares means corresponding tokens are attended to by the query.

blocks are arranged directly in 2D plate, as depicted in Fig. 6(b). As another example of sparse pattern on vision data, Axial Transformer (Ho et al., 2019) applies independent attention modules over each axis of the image. Each attention module mixes information along one axis while keeping information along the other axis independent, as illustrated in Fig. 6(c). This can be understood as horizontally and vertically flattening image pixels in raster-scan order and then applying strided attention with gaps of image width and height, respectively.

4.1.2. Content-based sparse attention

Another line of work creates a sparse graph based on input content, i.e., the sparse connections are conditioned on inputs.

A straightforward way of constructing a content-based sparse graph is to select those keys that are likely to have large similarity scores with the given query. To efficiently construct the sparse graph, we can recur to Maximum Inner Product Search (MIPS) problem, where one tries to find the keys with maximum dot product with a query without computing all dot product terms. Routing Transformer (Roy et al., 2021) uses k-means clustering to cluster both queries $\{\mathbf{q}_i\}_{i=1}^{T}$ and keys $\{\mathbf{k}_i\}_{i=}^{T}$ on the same set of centroid vectors $\{\boldsymbol{\mu}_i\}_{i=1}^{k}$. Each query only attends to the keys that belong to the same cluster. During training, the cluster centroid vectors are updated using the exponentially moving average of vectors assigned to it, divided by the exponentially moving average of cluster counts:

$$\tilde{\mu} \leftarrow \lambda\tilde{\mu} + (1-\lambda)\left(\sum_{i:\mu(\mathbf{q}_i)=\mu} \mathbf{q}_i + \sum_{j:\mu(\mathbf{k}_j)=\mu} \mathbf{k}_j\right), \tag{8}$$

$$c_\mu \leftarrow \lambda c_\mu + (1-\lambda)|\mu|, \tag{9}$$

$$\mu \leftarrow \frac{\tilde{\mu}}{c_\mu}, \tag{10}$$

where $|\mu|$ denotes the number of vectors currently in cluster μ and $\lambda \in (0,1)$ is a hyperparameter.

Let $\mathcal{P}_i$ denote the set of indices of keys that the ith query attend to. $\mathcal{P}_i$ in Routing Transformer is defined as

$$\mathcal{P}_i = \{j : \mu(\mathbf{q}_i) = \mu(\mathbf{k}_j)\}. \tag{11}$$

Reformer (Kitaev et al., 2020) uses locality-sensitive hashing (LSH) to select key–value pairs for each query. The proposed LSH attention allows each token to attend only to the tokens within the same hashing bucket. The basic idea is to use an LSH function to hash queries and keys into several buckets, with similar items fall in the same bucket with high probability. Specifically, they use the random matrix method for the LSH function. Let b be the number of buckets, given a random matrix R of size $[D_k, b/2]$, the LSH function is computed by :

$$h(x) = \arg\max([xR; -xR]). \tag{12}$$

The LSH attention allows the ith query to attend only to key–value pairs with indices

$$\mathcal{P}_i = \{j : h(\mathbf{q}_i) = h(\mathbf{k}_j)\}. \tag{13}$$

Sparse Adaptive Connection (SAC) (Li et al., 2020b) views the input sequence as a graph and learns to construct attention edges to improve task-specific performances using an adaptive sparse connection. SAC uses an LSTM edge predictor to construct edges between tokens. With no ground truth for edges, the edge predictor is trained with reinforcement learning.

Sparse Sinkhorn Attention (Tay et al., 2020b) first splits queries and keys into several blocks and assigns a key block to each query block. Each query is only allowed to attend to the keys in the key block that is assigned to its corresponding query block. The assignment of key blocks is controlled by a sorting network, which uses Sinkhorn normalization to produce a doubly stochastic matrix as the permutation matrix representing the assignment. They use this content-based block sparse attention along with block local attention introduced in Section 4.1.1 to enhance the ability of the model to model locality.

4.2. Linearized attention

Assuming $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{T\times D}$, the complexity of computing $\mathrm{softmax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}$ is quadratic w.r.t. sequence length T, as illustrated in Fig. 7(a). If $\mathrm{softmax}(\mathbf{Q}\mathbf{K}^\top)$ can be disentangled into $\mathbf{Q}'\mathbf{K}'^\top$, we can compute $\mathbf{Q}'\mathbf{K}'^\top\mathbf{V}$ in reversed order (i.e., $\mathbf{Q}'\left(\mathbf{K}'^\top\mathbf{V}\right)$), leading to a complexity of $\mathcal{O}(T)$.

Let $\hat{\mathbf{A}} = \exp(\mathbf{Q}\mathbf{K}^\top)$ denote un-normalized attention matrix, and $\exp(\cdot)$ is applied element-wise, the regular attention can be rewritten as $\mathbf{Z} = \mathbf{D}^{-1}\hat{\mathbf{A}}\mathbf{V}$, where $\mathbf{D} = \mathrm{diag}(\hat{\mathbf{A}}\mathbf{1}_T^\top)$; $\mathbf{1}_T^\top$ is the all-ones column vector of length T; $\mathrm{diag}(\cdot)$ is a diagonal matrix with the input vector as the diagonal.

Linearized attention is a class of methods that approximate or replace the unnormalized attention matrix $\exp(\mathbf{Q}\mathbf{K}^\top)$ with $\phi(\mathbf{Q})\phi(\mathbf{K})^\top$, where ϕ is a feature map that is applied in row-wise manner. Hence the computation of unnormalized attention matrix can be linearized by computing $\phi(\mathbf{Q})\left(\phi(\mathbf{K})^\top\mathbf{V}\right)$,[6] as illustrated in Fig. 7(b).

To gain further insights into linearized attention, we derive the formulation in vector form. We consider a general form of attention

$$\mathbf{z}_i = \sum_j \frac{\mathrm{sim}(\mathbf{q}_i, \mathbf{k}_j)}{\sum_{j'} \mathrm{sim}(\mathbf{q}_i, \mathbf{k}_{j'})}\mathbf{v}_j, \tag{14}$$

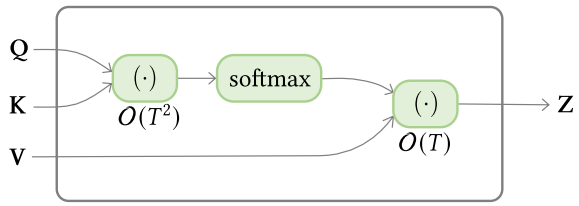
where $\mathrm{sim}(\cdot,\cdot)$ is a scoring function measuring similarity between input vectors. In vanilla Transformer, the scoring function is the exponential of inner product $\exp(\langle\cdot,\cdot\rangle)$. A natural choice of $\mathrm{sim}(\cdot,\cdot)$ is a kernel function $\mathcal{K}(\mathbf{x},\mathbf{y}) = \phi(\mathbf{x})\phi(\mathbf{y})^\top$, which leads to

$$\mathbf{z}_i = \sum_j \frac{\phi(\mathbf{q}_i)\phi(\mathbf{k}_j)^\top}{\sum_{j'} \phi(\mathbf{q}_i)\phi(\mathbf{k}_{j'})^\top}\mathbf{v}_j \tag{15}$$

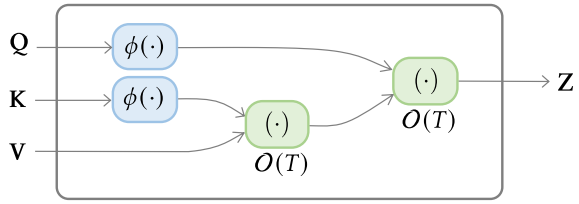
$$= \frac{\phi(\mathbf{q}_i)\sum_j \phi(\mathbf{k}_j)\otimes\mathbf{v}_j}{\phi(\mathbf{q}_i)\sum_{j'} \phi(\mathbf{k}_{j'})^\top}, \tag{16}$$

where $\otimes$ denotes outer product of vectors. Based on this formulation, attention can be linearized by first computing the highlighted terms

[6] Similarly, the partition term $\mathbf{D}$ can be computed with $\phi(\mathbf{Q})\left(\phi(\mathbf{K})^\top\mathbf{1}_T^\top\right)$ in linear time.



(a) standard self-attention



(b) linearized self-attention

Fig. 7. Illustration of complexity difference between standard self-attention and linearized self-attention.

$\sum_j \phi(\mathbf{k}_j) \otimes \mathbf{v}_j$ and $\sum_{j'} \phi(\mathbf{k}_{j'})^\top$. This could be especially beneficial for autoregressive attention, as the cumulative sums $\mathbf{S}_i = \sum_{j=1}^{i} \phi(\mathbf{k}_j) \otimes \mathbf{v}_j$ and $\mathbf{u}_i = \sum_{j=1}^{i} \phi(\mathbf{k}_j)$ can be computed from $\mathbf{S}_{i-1}$ and $\mathbf{u}_{i-1}$ in constant time. The effectively enables Transformer decoders to run like RNNs.

An interpretation of Eq. (16) is that the model maintains a *memory matrix* by aggregating *associations* represented by outer products of (feature mapped) keys and values, and then retrieve a value by multiplying the memory matrix with feature mapped query with proper normalization. There are two key components in this approach: (1) feature map $\phi(\cdot)$, and (2) aggregation rule.

4.2.1. Feature maps

Linear Transformer (Katharopoulos et al., 2020) propose to use a simple feature map $\phi_i(\mathbf{x}) = \mathrm{elu}(x_i) + 1$. This feature map does not aim to approximate dot product attention, but is empirically proved to perform on par with the standard Transformer.

Performer (Choromanski et al., 2020a,b) uses random feature maps that approximate the scoring function of Transformer. The random feature maps take functions $f_1, \dots, f_l : \mathbb{R} \to \mathbb{R}$ and $h : \mathbb{R}^D \to \mathbb{R}$.

$$\phi(\mathbf{x}) = \frac{h(\mathbf{x})}{\sqrt{m}}[f_1(\omega_1^\top \mathbf{x}), \dots, f_m(\omega_m^\top \mathbf{x}), \dots, f_l(\omega_1^\top \mathbf{x}), \dots, f_l(\omega_m^\top \mathbf{x})], \tag{17}$$

where $\omega_1, \dots, \omega_m \overset{\text{iid}}{\sim} \mathcal{D}$ are drawn from some distribution $\mathcal{D} \in \mathcal{P}(\mathbb{R}^D)$.

The first version of Performer (Choromanski et al., 2020a) is inspired from the random Fourier feature map (Rahimi and Recht, 2007) that was originally used to approximate Gaussian kernel. It uses trigonometric functions with $h(\mathbf{x}) = \exp(\frac{\|\mathbf{x}\|^2}{2}), l = 2, f_1 = \sin, f_2 = \cos$. This approach has also been used in Random Feature Attention (RFA) (Peng et al., 2021), with the difference that $h(\mathbf{x})$ is set to 1 as the queries and keys are ℓ_2-normalized before applying the feature map.

Although the trigonometric random feature map leads to an unbiased approximation, it does not guarantee non-negative attention scores and thus could lead to unstable behaviors and abnormal behaviors. To mitigate this issue, the second version of Performer (Choromanski et al., 2020b) proposes positive random feature maps, which uses $h(\mathbf{x}) = \exp(-\frac{\|\mathbf{x}\|^2}{2}), l = 1, f_1 = \exp$ and thus guarantees unbiased and non-negative approximation of dot-product attention. This approach is more stable than Choromanski et al. (2020a) and reports better approximation results.

In addition to using random feature maps to approximate standard dot product attention, Peng et al. (2021) and Choromanski et al. (2020b) also explore approximating order-1 arc-cosine kernel with $h(\mathbf{x}) = 1, l = 1, f_1 = \mathrm{ReLU}$. This feature map has been show to be effective in various tasks including machine translation and protein sequence modeling.

Schlag et al. (2021) design a feature map that aims at facilitating orthogonality in feature space. Specifically, given an input $\mathbf{x} \in \mathbb{R}^D$, the feature map $\phi : \mathbb{R}^D \to \mathbb{R}^{2\nu D}$ is defined by the partial function

$$\phi_{i+2(j-1)D}(\mathbf{x}) = \mathrm{ReLU}([\mathbf{x}, -\mathbf{x}])_i \mathrm{ReLU}([\mathbf{x}, -\mathbf{x}])_{i+j} \quad \text{for } i = 1, \dots, 2D, j = 1, \dots, \nu. \tag{18}$$

4.2.2. Aggregation rule

In Eq. (16) the associations $\{\phi(\mathbf{k})_j \otimes \mathbf{v}_j\}$ are aggregated into the memory matrix by simple summation. This is adopted by several studies (Katharopoulos et al., 2020; Choromanski et al., 2020a,b). However, it could be more beneficial for the network to selectively drop associations as new associations are added to the memory matrix.

RFA (Peng et al., 2021) introduces a gating mechanism to the summation to model local dependency in sequence data. Specifically, when adding a new association to the memory matrix $\mathbf{S}$, at a particular time step, they weigh $\mathbf{S}$ by a learnable, input-dependent scalar g, and the new association by $(1-g)$ (and a similar mechanism to $\mathbf{u}$). With this modification, history associations are exponentially decayed and recent context is favored in each timestep.

Schlag et al. (2021) argue that simple summation limits the capacity of the memory matrix and thus propose to enlarge the capacity in a write-and-remove fashion. Specifically, given a new input key–value pair $(\mathbf{k}_i, \mathbf{v}_i)$, the model first retrieve the value $\bar{\mathbf{v}}_i$ currently associated with $\mathbf{k}_i$ using matrix multiplication. It then writes to the memory matrix a convex combination of $\bar{\mathbf{v}}_i$ and $\mathbf{v}_i$, using a input-dependent gating scalar g, and removes the association $\bar{\mathbf{v}}_i$. They also propose *sum normalization* (normalizing $\phi(\mathbf{q}_i), \phi(\mathbf{k}_i)$ by the sum of their components before updating the memory matrix) instead of normalizing with the denominator in Eq. (16) for this aggregation rule.

4.3. Query prototyping and memory compression

Apart from using sparse attention or kernel-based linearized attention, one could also reduce the complexity of attention by reducing the number of queries or key–value pairs, which leads to *query prototyping* and *memory compression*[7] methods, respectively.

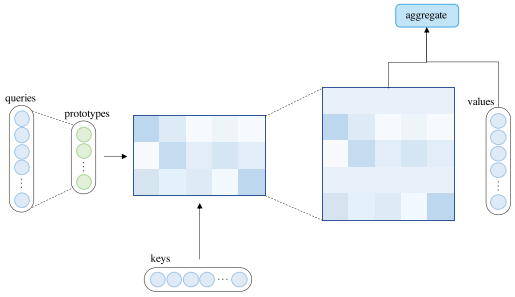
4.3.1. Attention with prototype queries

In query prototyping, several prototypes of queries serve as the main source to compute attention distributions. The model either copies the distributions to the positions of represented queries or filling those positions with discrete uniform distributions. Fig. 8(a) illustrates the computing flow of query prototyping.

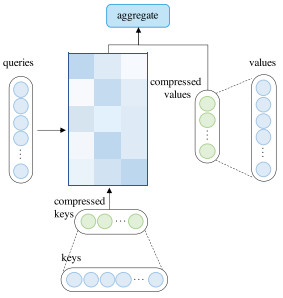
Clustered Attention (Vyas et al., 2020) groups queries into several clusters and then computes attention distributions for cluster centroids. All queries in a cluster share the attention distribution calculated with the corresponding centroid.

Informer (Zhou et al., 2021) selects prototypes from queries using explicit query sparsity measurement, which is derived from an approximation of the Kullback–Leibler divergence between the query's attention distribution and the discrete uniform distribution. Attention distributions are then only calculated for the top-u queries under query sparsity measurement. The rest of the queries are assigned with discrete uniform distributions.

[7] The key–value pairs are often referred to as a key–value memory (hence the name memory compression).



(a) Query prototyping



(b) Memory compression

Fig. 8. Query prototyping and memory compression.

4.3.2. Attention with compressed key–value memory

Apart from decreasing the number of queries with query prototyping, one can also reduce the complexity by reducing the number of the key–value pairs before applying the attention mechanism, as depicted in Fig. 8(b).

Liu et al. (2018) propose Memory Compressed Attention (MCA) that reduces the number of keys and values using a strided convolution. This modification is used as a complement to local attention proposed in the same work (as discussed in Section 4.1), in that it can capture global context. The mechanism reduces the number of keys and values by a factor of kernel size k and thus allowing to process significantly longer sequences than vanilla Transformer given the same computation resources.

Set Transformer (Lee et al., 2019) and Luna (Ma et al., 2021) use a number of external trainable global nodes to summarize information from inputs and then the summarized representations serve as a compressed memory that the inputs attend to. This reduces the quadratic complexity of self-attention to linear complexity w.r.t. sequence length.

Linformer (Wang et al., 2020a) utilizes linear projections to project keys and values from length n to a smaller length n_k. This also reduces the complexity of self-attention to linear. The drawback of this approach is that an input sequence length has to be assumed and hence it cannot be used in autoregressive attention.

Poolingformer (Zhang et al., 2021) adopts two-level attention that combines a sliding window attention and a compressed memory attention. The compressed memory module is used after the sliding window attention to increase the receptive field. They explore a few different pooling operations as the compression operation to compress the number of keys and values, including max pooling and pooling with Dynamic Convolution (Wu et al., 2019).

4.4. Low-rank self-attention

Some empirical and theoretical analyses (Guo et al., 2019b; Wang et al., 2020a) report the self-attention matrix $\mathbf{A} \in \mathbb{R}^{T\times T}$ is often low-rank.[8] The implications of this property are twofold: (1) The low-rank property could be explicitly modeled with parameterization; (2) The self-attention matrix could be replaced by a low-rank approximation.

[8] The rank of $\mathbf{A}$ is far lower than input length T.

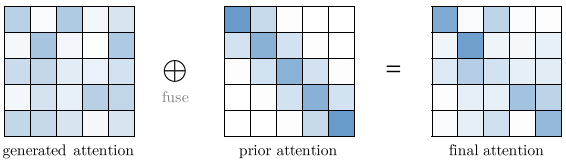


Fig. 9. Attention with prior. This type of model fuse generated attention scores with prior attention scores, producing the final attention scores for attention computation.

4.4.1. Low-rank parameterization

The fact that the rank of the attention matrix is less than sequence length implies that, for scenarios where the inputs are typically short, setting $D_k > T$ would be more than an over-parameterization and lead to overfitting. It is thus reasonable to limit the dimension of D_k to explicitly model the low-rank property as an inductive bias. Guo et al. (2019b) decompose self-attention matrix into a low-rank attention module with small D_k that captures long-range non-local interactions, and a band attention module that captures local dependencies.

4.4.2. Low-rank approximation

Another implication of the low-rank property of the attention matrix is that one can use a low-rank matrix approximation to reduce the complexity of self-attention. A closely related methodology is the low-rank approximation of kernel matrices. We believe some existing works are inspired by kernel approximation.

Some of the aforementioned linearized attention methods in Section 4.2 are inspired from kernel approximation with random feature maps. For example, Performer (Choromanski et al., 2020a) follows the Random Fourier feature map originally proposed to approximate Gaussian kernels. The method first decomposes the attention distribution matrix $\mathbf{A}$ into $\mathbf{C}_Q\mathbf{G}\mathbf{C}_K$ where $\mathbf{G}$ is a Gaussian kernel matrix and the random feature map is used to approximate $\mathbf{G}$.

Another line of work follow the idea of Nyström method. These Nyström-based methods (Chen et al., 2020a; Xiong et al., 2021) first select m landmark nodes from the T inputs with down-sampling methods (e.g., strided average pooling). Let $\tilde{\mathbf{Q}}, \tilde{\mathbf{K}}$ be the selected landmark queries and keys, then the follow approximation is used in the attention computation

$$\tilde{\mathbf{A}} = \mathrm{softmax}\left(\mathbf{Q}\tilde{\mathbf{K}}^\top\right)\left(\mathrm{softmax}\left(\tilde{\mathbf{Q}}\tilde{\mathbf{K}}^\top\right)\right)^{-1}\mathrm{softmax}\left(\tilde{\mathbf{Q}}\mathbf{K}^\top\right). \tag{19}$$

Note that $\mathbf{M}^{-1} = \left(\mathrm{softmax}\left(\tilde{\mathbf{Q}}\tilde{\mathbf{K}}^\top\right)\right)^{-1}$ in Eq. (19) does not always exist. To mitigate this issue, CSALR (Chen et al., 2020a) adds an identity matrix to $\mathbf{M}$ to make sure that the inverse always exists. Nyströmformer (Xiong et al., 2021) uses the Moore–Penrose pseudoinverse of $\mathbf{M}$ instead of the inverse so that the approximation can be made for cases where $\mathbf{M}$ is singular.

4.5. Attention with prior

Attention mechanism generally outputs an expected attended value as a weighted sum of vectors, where the weights are an attention distribution over the values. Traditionally, the distribution is generated from inputs (e.g., $\mathrm{softmax}(\mathbf{QK}^\top)$ in vanilla Transformer). As a generalized case, attention distribution can also come from other sources, which we refer to as *prior*. Prior attention distribution can be a supplement or substitute for distribution generated from inputs. We abstract this formulation of attention as *attention with prior*, as depicted in Fig. 9. In most cases, the fusion of two attention distribution can be done by computing a weighted sum of the scores corresponding to the prior and generated attention before applying softmax.

4.5.1. Encoding structural information

Attention mechanism is often interpreted as aggregating information from inputs based on weights generated from content similarity measures. However, it might be insufficient to base the weights solely on the content of input elements. Since the attention matrix encodes relations between inputs, it is natural to encode structural information using prior attention.

For instance, one can use prior attention to encode positional relations between input elements. The prior attention can be formulated as a trainable attention prior $\mathbf{B}$ that is added directly to the unnormalized attention matrix (Raffel et al., 2020) or generated from position embeddings (Ke et al., 2020).

One can also use prior attention to model more complex structural information (e.g., edges in graph data). Graphormer (Ying et al., 2021a) utilizes a spatial encoding $\mathbf{B}$ that encodes connectivity between nodes, as well as an edge encoding $\mathbf{C}$ that encodes edge features.

4.5.2. Prior that models locality

Some types of data (e.g., text) can exhibit a strong preference for the locality. This property can be explicitly encoded as a prior attention. A simple method would be to use a Gaussian distribution over positions. Specifically, one could multiply the generated attention distribution with some Gaussian density and then renormalize, which is equivalent to adding to the generated attention scores $\mathbf{A}$ a bias term $\mathbf{G}$, where higher G_{ij} indicates a higher prior probability that the ith input attend to the jth input.

Yang et al. (2018) proposes to first predict a central position p_i for each $\mathbf{q}_i$ using a simple feed-forward network. The Gaussian bias is then defined to be

$$G_{ij} = -\frac{(j-p_i)^2}{2\sigma^2}, \tag{20}$$

where σ denotes standard deviation for the Gaussian and can be determined as a hyperparameter or predicted from inputs.

Gaussian Transformer (Guo et al., 2019c) assumes the central position to be i for each $\mathbf{q}_i$ and defines the bias to bes

$$G_{ij} = -|w(i-j)^2 + b|, \tag{21}$$

where $w \geq 0, b \leq 0$ are scalar parameters that controls the deviation and reduce the weight for central position, respectively.

4.5.3. Prior from lower modules

In Transformer architecture, it is often observed the attention distributions are similar in adjacent layers. It is thus natural to provide attention distribution from previous layer as a prior for attention computation. The final attention scores can be defined as

$$\hat{\mathbf{A}}^{(l)} = w_1 \cdot \mathbf{A}^{(l)} + w_2 \cdot g(\mathbf{A}^{(l-1)}), \tag{22}$$

where $\mathbf{A}^{(l)}$ denotes the attention scores of the lth layer, $w_1, w_2 \in \mathbb{R}$ are weight applied to the scores from adjacent layers, and $g : \mathbb{R}^{n\times n} \to \mathbb{R}^{n\times n}$ is a function that translate previous scores to the prior to be applied.

Predictive Attention Transformer (Wang et al., 2021b) proposes to apply a 2D-convolutional layer to previous attention scores and compute the final attention scores as a convex combination of the generated attention scores and the convolved scores. This is equivalent to setting $w_1 = \alpha, w_2 = 1-\alpha$ and $g(\cdot)$ to be a convolutional layer in Eq. (22). They experiment training such a model from scratch and finetune after adapting the pre-trained BERT model, and both sets of experiments show improvements over baseline models.

Realformer (He et al., 2020b) uses adds the previous attention scores directly to the generated attention scores, thus resembles a residual skip connection on attention maps. It is equivalent to setting $w_1 = w_2 = 1$ and $g(\cdot)$ to be identity map in Eq. (22). They conduct pre-training experiments on this model. The results show that this model outperforms the baseline BERT model in multiple datasets and surpasses the baseline model even when pre-training budgets are significantly lower.

As an extreme case, Lazyformer (Ying et al., 2021b) proposes to share attention maps between a number of adjacent layers. This is equivalent to setting $g(\cdot)$ to identity and switch the settings of $w_1 = 0, w_2 = 1$ and $w_1 = 1, w_2 = 0$ alternatively. The benefit of this approach is that the attention maps are computed only once and reused several times in the succeeding layers, thus reducing the computation cost. Their pre-training experiments show that the resulting model remains effective while being much more efficient to compute.

4.5.4. Prior as multi-task adapters

Adapters are task-dependent, trainale modules that are attached in specific locations of a pre-trained network for cross-task efficient parameter sharing (Rebuffi et al., 2017). Pilault et al. (2021) propose a Conditionally Adaptive Multi-Task Learning (CAMTL) framework that uses a trainable attention prior $M(\mathbf{z}_i)$ that depends on task encoding $\mathbf{z}_i \in \mathbb{R}^{D_z}$

$$M(\mathbf{z}_i) = \bigoplus_{j=1}^{m} A'_j(\mathbf{z}_i), \quad A'_j(\mathbf{z}_i) = A_j \gamma_i(\mathbf{z}_i) + \beta_i(\mathbf{z}_i), \tag{23}$$

where $\bigoplus$ denotes direct sum, $A_j \in \mathbb{R}^{(n/m)\times(n/m)}$ are trainable parameters, and $\gamma_j, \beta_j : \mathbb{R}^{D_z} \to \mathbb{R}^{(n/m)\times(n/m)}$ are Feature Wise Linear Modulation functions (Perez et al., 2018). A maximum sequence length n_{max} is specified in implementation. The prior is formulated as a block diagonal matrix and added to the attention scores of upper layers in pre-trained Transformers to serve as an adapter for parameter-efficient multi-task inductive knowledge transfer.

4.5.5. Attention with only prior

Some works have explored using an attention distribution that is independent of pair-wise interaction between inputs. In other words, their models exploit only a prior attention distribution.

Zhang et al. (2018) design an efficient Transformer decoder variant called average attention network that uses a discrete uniform distribution as the sole source of attention distribution. The values are thus aggregated as a cumulative-average of all values. To improve the expressiveness of the network, they further adds a feed-forward gating layer on top of the average attention module. The advantage of this approach is that the adapted Transformer decoder can train in a parallel manner as usual Transformers do and decode like an RNN, thus avoiding the $\mathcal{O}(T^2)$ complexity in decoding.

You et al. (2020) utilize a Gaussian distribution as the hardcoded attention distribution for attention calculation. The intuition is very similar to Yang et al. (2018) and Guo et al. (2019c) in that attention distribution should be focused on a certain local window. Distinctively, they drop the generated attention completely and use only the Gaussian distribution for attention computation. In this approach, the mean (central position) and variance are designed to be hyperparameters. The experiments show that the hardcoded attention, when applied only to self-attention, can achieve comparable performance to the baseline model in machine translation tasks.

Synthesizer (Tay et al., 2020a) proposes to replace generated attention scores with: (1) a learnable, randomly initialized attention scores, and (2) attention scores output by a feed-forward network that is only conditioned on the querying input itself. The experiments on machine translation and language modeling show that these variants can achieve competitive performance with vanilla Transformer. It is not explained why these variants work but the empirical results are intriguing.

4.6. Improved multi-head mechanism

Multi-head attention is appealing for the ability to jointly attend to information from different representation subspaces at different positions. However, there is no mechanism to guarantee that different attention heads indeed capture distinct features.

4.6.1. Head behavior modeling

A basic motivation for using multi-head attention is to allow the model to jointly attend to information from different representation subspaces at different positions (Vaswani et al., 2017). However, in vanilla Transformer there is no explicit mechanism to guarantee different behavior across attention heads, nor is there any mechanism for heads to interact with each other. A line of work is dedicated to improving multi-head mechanism by introducing incorporating more sophisticated mechanisms that guide the behavior of different attention heads or allow interaction across attention heads.

Li et al. (2018) introduce an auxiliary disagreement regularization term into loss function to encourage diversity among different attention heads. Two regularization terms are respectively to maximize cosine distances of the input subspaces and output representations, while the last one is to disperse the positions attended by multiple heads with element-wise multiplication of the corresponding attention matrices.

Several probing works have revealed that pre-trained Transformer models exhibit certain patterns of self-attention that are of little linguistic backing. As a representative work, Kovaleva et al. (2019) identify several simple attention patterns in BERT. For instance, many of the attention heads simply pay attention to special BERT tokens `[CLS]` and `[SEP]`. As a result, some constraints can be introduced to boost the training of Transformer models. To this end, Deshpande and Narasimhan (2020) propose to use an auxiliary loss, which is defined to be the Frobenius norm between attention distribution maps and predefined attention patterns.

Talking-head Attention (Shazeer et al., 2020) uses a talking head mechanism that linearly projects the generated attention scores from h_k to h heads, applies softmax in that space, and then projects to h_v heads for value aggregation. The motivation is to encourage the model to move information between attention heads in a learnable fashion.

Collaborative Multi-head Attention (Cordonnier et al., 2020) uses shared query and key projection $\mathbf{W}^Q$ and $\mathbf{W}^K$ and a mixing vector $\mathbf{m}_i$ for the ith head to filter from the projection parameters such that Eq. (3) is adapted to

$$\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}^Q \text{diag}(\mathbf{m}_i), \mathbf{K}\mathbf{W}^K, \mathbf{V}\mathbf{W}_i^V), \tag{24}$$

where $\mathbf{W}^Q$ and $\mathbf{W}^K$ are shared by all the attention heads.

4.6.2. Multi-head with restricted spans

Vanilla attention adopts full attention spans assume, where a query can attend to all of the key–value pairs. However, it is often observed that some heads focus their attention distribution mainly in a local context while some other heads attend to broader contexts. It could thus be beneficial to restrict the attention spans:

- *Locality*. Restricting attention spans induce explicit local constraints. This is advantageous in cases where locality is an important prior.
- *Efficiency*. If implemented appropriately, such a model can scale to very long sequences without introducing additional memory footprint and computational time.

Restricting attention spans can be expressed as multiplying each attention distribution value with a mask value and then re-normalize, where the mask can be expressed as a non-increasing function that maps a distance to a value in $[0, 1]$. A vanilla attention assigns a mask value of 1 for all distances, as depicted in Fig. 10(a).

Sukhbaatar et al. (2019a) propose to use a learnable attention span, as depicted in Fig. 10(b) . The mask is parameterized by a learnable scalar z and a hyperparameter R. The experiments on character-level language modeling show that the adaptive-span models outperform baseline models while having significantly fewer FLOPS. It is also observed that lower layers generally have smaller learned spans and higher layers otherwise. This indicates that the model can learn a hierarchical composition of features.

Multi-Scale Transformer (Guo et al., 2020) proposes to use a fixed attention span, with different heads in different layers using a different max span. The fixed attention span is depicted in Fig. 10(c). The attention is restricted within a fixed window which is controlled by a *scale* value w. They design the scales from an intuitive linguistic perspective and empirical observation from BERT such that higher layers tend to have more large scales (e.g., large span size), and lower layers should be confined with a smaller scale. Their experiments on several tasks show that the model can outperform baseline models while accelerating inference on long sequences.

4.6.3. Multi-head with refined aggregation

After each attention head computes its output representation, the vanilla multi-head attention (Vaswani et al., 2017) concatenates these representation and then apply a linear transformation to the concatenated representation to obtain the final output representation, as formulated in Eq. (2). Combining Eq. (1)(2) and (3), one can see that this *concatenate-and-project* formulation is equivalent to summation over H re-parameterized attention outputs. To this end, we first divide $\mathbf{W}^O \in \mathbb{R}^{D_m \times D_m}$ into H blocks

$$\mathbf{W}^O = [\mathbf{W}_1^O; \mathbf{W}_2^O; \cdots; \mathbf{W}_H^O], \tag{25}$$

where each $\mathbf{W}_i^O$ is of dimension $D_v \times D_m$. It is thus easy to see that multi-head attention can be reformulated as

$$\text{MultiHeadAttn}(Q, K, V) = \sum_{i=1}^{H} \text{Attention}(Q\mathbf{W}_i^Q, K\mathbf{W}_i^K, V\mathbf{W}_i^V \mathbf{W}_i^O). \tag{26}$$

One might argue that this simple *aggregate-by-summation* paradigm does not fully exploit the expressiveness of multi-head attention and that it is more desirable to use a more complex aggregation.

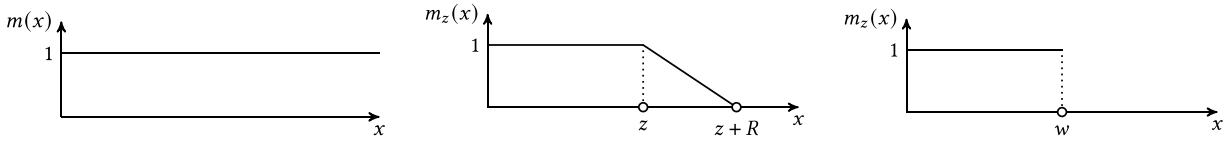
Li et al. (2019b), Gu and Feng (2019) propose to use routing methods, originally proposed for capsule networks (Sabour et al., 2017), to further aggregate information produced by different attention heads. The outputs of attention heads are first transformed into input capsules, then output capsules are obtained after the iterative routing process. The output capsules are then concatenated as a final output of multi-head attention. These two works both utilizes two routing mechanisms, namely *dynamic routing* (Sabour et al., 2017) and *EM routing* (Hinton et al., 2018). One would notice that iterative routing introduces additional parameters and computational overhead. Li et al. (2019b) empirically show that applying the routing mechanism only to the lower layers can best balance the translation performance and computational efficiency.

4.6.4. Other modifications

Several other modifications to the multi-head mechanism have been proposed to improve multi-head attention.

Shazeer (2019) propose multi-query attention, where key–value pairs are shared among attention heads (i.e., to use only one key projection and one value projection for all attention heads). The advantage of this method is that it reduces the memory bandwidth requirements for decoding and results in a model that is faster to decode, while incurring only minor quality degradation from the baseline.

Bhojanapalli et al. (2020) establish that small attention key size can affect its ability to represent arbitrary distribution. They thus propose to disentangle head size from the number of heads h, as opposed to the common practice that sets the head size to be D_m/h. It is observed empirically that setting attention head size to be input sequence length is beneficial.



(a) mask function for vanilla attention (b) mask function for adaptive span (c) mask function for fixed span

Fig. 10. Three types of span masking function $m(x)$. The horizontal axis represents distance x and vertical axis the mask value.

5. Other module-level modifications

5.1. Position representations

Definition 5.1 (*permutation equivariant function*)**.** Let Π_n be the set of all permutations of indices $\{1, 2, \dots, T\}$. A function $f : \mathcal{X}^T \to \mathcal{Y}^T$ is said to be *permutation equivariant* if and only if for any $\pi \in \Pi_T$

$$f(\pi x) = \pi f(x). \tag{27}$$

It is easy to verify that Convolution and Recurrence networks are not permutation equivariant. However, both self-attention modules and position-wise feed-forward layers in Transformer are permutation equivariant, which could be a problem when it comes to modeling problems other than *set-input* problems where the structure of inputs is needed. For example, when modeling sequences of text, the ordering of words matters and it is thus crucial to properly encode the positions of words in Transformer architecture. Therefore, additional mechanisms are required to inject positional information into Transformers. A common design is to first represent positional information using vectors and then infuse the vectors to the model as an additional input.

5.1.1. Absolute position representations

In vanilla Transformer (Vaswani et al., 2017), positional information is encoded as absolute sinusoidal position encodings. For each position index t, the encoding is a vector $\mathbf{p}_t = \mathrm{PE}(t) \in \mathbb{R}^{D_m}$, of which every element is a sinusoidal (sin/cos) function of the index with pre-defined frequency.

$$\mathrm{PE}(t)_i = \begin{cases} \sin(\omega_i t) & \text{if } i \text{ is even,} \\ \cos(\omega_i t) & \text{if } i \text{ is odd,} \end{cases} \tag{28}$$

where ω_i is the hand-crafted frequency for each dimension. The position encoding of each position in the sequence is then added to the token embeddings and fed to Transformer.

Another way of representing absolute positions is to learn a set of positional embeddings for each position (Gehring et al., 2017; Devlin et al., 2019). Compared to hand-crafted position representation, learned embeddings are more flexible in that position representation can adapt to tasks through back-propagation. But the number of embeddings is limited up to a maximum sequence length determined before training, which makes this approach no longer *inductive*, i.e., not able to handle sequences longer than sequences seen in the training time (Liu et al., 2020b; Chu et al., 2021).

Wang et al. (2021a) propose to use sinusoidal position representation, but with each frequency ω_i (in Eq. (28)) learned from data. This approach retains inductiveness but is more flexible than hand-crafted sinusoidal encoding. FLOATER (Liu et al., 2020b) frames positional representation as a continuous dynamical system and adopts Neural ODE to enable end-to-end training with backpropagation. This method is inductive and flexible while being parameter efficient compared to a fully learnable approach.

The Vanilla approach to incorporating absolute position representations is to add position encodings/embeddings to token embeddings. However, as the input signals propagate through the layers, the positional information might get lost in the upper layers. Later works find it beneficial to add position representations to inputs to each Transformer layer (Al-Rfou et al., 2019; Dehghani et al., 2019; Liu et al., 2020b; Guo et al., 2019b).

5.1.2. Relative position representations

Another line of works focuses on representing positional relationships between tokens instead of positions of individual tokens. The intuition is that in self-attention, pairwise positional relationships between input elements (direction and distance) could be more beneficial than positions of elements. Methods following this principles are called relative positional representation. Shaw et al. (2018) propose to add a learnable relative position embedding to keys of attention mechanism

$$\mathbf{k}'_j = \mathbf{k}_j + \mathbf{r}_{ij}, \text{ for } i = 1, \dots, n, \tag{29}$$

$$\mathbf{r}_{ij} = \mathbf{R}_{\mathrm{clip}(i-j)}, \tag{30}$$

$$\mathrm{clip}(x) = \max(-K, \min(x, K)), \tag{31}$$

where $\mathbf{r}_{ij} \in \mathbb{R}^{D_k}$ is the relative position embedding for relation between position i and j and K is the largest offset that determines the number of embedding. Typically K is set to a length that can accommodate most input sequences. As a special case, InDIGO (Gu et al., 2019) sets K to 3 for their specially designed framework for non-autoregressive generation. As an incremental effort, Music Transformer (Huang et al., 2019) further introduce a mechanism to reduce the intermediate memory requirements for this approach. Similar to this approach, T5 (Raffel et al., 2020) adopt a simplified form of relative position embeddings where each embedding is only a learnable scalar that is added to the corresponding score used for computing the attention weights.

Transformer-XL (Dai et al., 2019) use a sinusoidal encoding to represent positional relationships but fuses contents and position information by redesign the computation of attention scores[9]

$$\mathbf{A}_{ij} = \mathbf{q}_i \mathbf{k}_j^\top + \mathbf{q}_i \left(\mathbf{R}_{i-j} \mathbf{W}^{K,R}\right)^\top + \mathbf{u}^1 \mathbf{k}_j^\top + \mathbf{u}^2 \left(\mathbf{R}_{i-j} \mathbf{W}^{K,R}\right)^\top, \tag{32}$$

where $\mathbf{W}^{K,R} \in \mathbb{R}^{D_m \times D_k}, \mathbf{u}^1, \mathbf{u}^2 \in \mathbb{R}^{D_k}$ are learnable parameters and $\mathbf{R}$ is a sinusoidal encoding matrix similar to position encoding in vanilla Transformer. Then softmax function is applied to scores $\mathbf{A}$ to provide attention weights. Note that the learnable sinusoidal encoding (Wang et al., 2021a) is also a drop-in replacement to hand-crafted $\mathbf{R}$.

DeBERTa (He et al., 2020a) utilizes position embeddings like Shaw et al. (2018) and applies the embeddings to the model in a disentangled style similar to Transformer-XL (Dai et al., 2019)

$$\mathbf{A}_{ij} = \mathbf{q}_i \mathbf{k}_j^\top + \mathbf{q}_i \left(\mathbf{r}_{ij} \mathbf{W}^{K,R}\right)^\top + \mathbf{k}_j \left(\mathbf{r}_{ij} \mathbf{W}^{Q,R}\right)^\top, \tag{33}$$

where $\mathbf{W}^{K,R}, \mathbf{W}^{Q,R} \in \mathbb{R}^{D_m \times D_k}$ are learnable parameters and $\mathbf{r}_{ij}$ is the learnable relative positional embedding as in Eq. (30). The first term is interpreted as a content-to-content attention, and the latter two terms are interpreted as (relative) content-to-position and position-to-content attention, respectively.

5.1.3. Other representations

Some research studies have explored using hybrid positional representations that contains both absolute and relative positional information. Transformer with Untied Position Encoding (TUPE) (Ke et al., 2020) re-designs the computation of attention scores as a combination of a content-to-content term, an absolute position-to-position term and a bias term representing relative positional relationships

$$\mathbf{A}_{ij} = \mathbf{q}_i \mathbf{k}_j^\top + \left(\mathbf{p}_i \mathbf{W}^{Q,P}\right) \left(\mathbf{p}_j \mathbf{W}^{K,P}\right)^\top + b_{j-i}, \tag{34}$$

[9] The scaling factor is omitted without loss of generality.

where $\mathbf{W}^{K,P}, \mathbf{W}^{Q,P} \in \mathbb{R}^{D_m \times D_k}$ are learnable parameters, $\mathbf{p}_i, \mathbf{p}_j$ are the position embeddings for positions i, j, and b_{j-i} is a learnable scalar relative position embedding.

One can also design a single set of positional representations that express both absolute and relative information. Roformer (Su et al., 2021) uses Rotary Position Embedding (RoPE) to represent the position of a token by multiplying the affine-transformed embedding of the tth input x_t by a rotatory matrix $\mathbf{R}_{\Theta,t}$

$$\mathbf{q}_t = \mathbf{x}_t \mathbf{W}^Q \mathbf{R}_{\Theta,t} \quad \mathbf{k}_t = \mathbf{x}_t \mathbf{W}^K \mathbf{R}_{\Theta,t}, \tag{35}$$

$$\mathbf{R}_{\Theta,t} = \bigoplus_{j=1}^{D_k/2} \mathbf{M}(t, \theta_j), \tag{36}$$

where $\bigoplus$ denotes *direct sum* of matrices. Each $\mathbf{M}(t, \theta_j)$ is a 2-D clockwise rotatory matrix of angle $t \cdot \theta_j$

$$\mathbf{M}(t, \theta_j) = \begin{bmatrix} \cos(t \cdot \theta_j) & \sin(t \cdot \theta_j) \\ -\sin(t \cdot \theta_j) & \cos(t \cdot \theta_j) \end{bmatrix}. \tag{37}$$

The key advantage of this formulation is that the induced representation is translation invariant, i.e., the attention score of $(\mathbf{q}_i, \mathbf{k}_j)$ is only related to their relative position offset

$$\mathbf{q}_i \mathbf{k}_j^\top = \left(\mathbf{x}_i \mathbf{W}^Q\right) \mathbf{R}_{\Theta,j-i} \left(\mathbf{x}_j \mathbf{W}^K\right)^\top. \tag{38}$$

In practice, the embedding matrix multiplication can be implemented by two element-wise multiplication for lower memory footprint. The RoPE uses the form of absolute embedding but can capture relative positional relations. This approach is compatible with linearized attention in Section 4.2.

5.1.4. Position representations without explicit encoding

Instead of explicitly introducing additional positional encodings, Wang et al. (2020b) propose to encode positional information in word embeddings, by generalizing embedding to continuous (complex-valued) functions over positions.

R-Transformer (Wang et al., 2019b) model locality of sequential data with a local RNN. Specifically, inputs to each block of R-Transformer are first fed to a local RNN and then to multi-Head self-attention module. The RNN structure introduces ordering information and captures local dependencies as a complement to self-attention.

Conditional positional encoding (CPE) (Chu et al., 2021) generate conditional position encodings at each layer for ViT with a 2-D convolution with zero-paddings. The intuition behind this approach is that convolution networks can implicitly encode absolute positional information with zero-paddings (Islam et al., 2020).

5.1.5. Position representation on Transformer decoders

It is worth noticing that masked self-attention is not permutation equivariant (Tsai et al., 2019). Thus a model that exploits only the decoder of Transformer has the potential of sensing positional information without incorporating explicit positional representation. This is confirmed by some empirical results on language modeling tasks (Irie et al., 2019; Schlag et al., 2021), where the authors find that removing position encodings even improves performance.

5.2. Layer normalization

Layer Normalization (LN), along with residual connection, is considered as a mechanism to stabilizing training of deep networks (e.g., alleviating ill-posed gradients and model degeneration). There are some studies that are dedicated to analyzing and improving LN module.

5.2.1. Placement of layer normalization

In vanilla Transformer, the LN layer lies between the residual blocks, called post-LN (Wang et al., 2019a). Later Transformer implementations (Vaswani et al., 2018; Klein et al., 2017) place the LN layer inside the residual connection before the attention or FFN, with an additional LN after the final layer to control the magnitude of final outputs, which is referred to as pre-LN.[10] The pre-LN has been adopted by numerous following research studies and implementations, e.g., (Baevski and Auli, 2019; Child et al., 2019; Wang et al., 2019a).

Xiong et al. (2020) theoretically investigate the gradients of Transformers and find that the gradients near the output layer are large at initialization in post-LN Transformers, which could be the reason why post-LN Transformers without learning rate warm-up (Vaswani et al., 2017)[11] leads to unstable training, whereas pre-LN Transformers do not suffer from the same problem. They thus deduce and empirically verify that warm-up stage can be safely removed for pre-LN Transformers.

Although Post-LN often results in unstable training and divergence, it usually outperforms pre-LN variants after convergence (Liu et al., 2020a). Similar to Xiong et al. (2020), Liu et al. (2020a) conduct theoretical and empirical analysis and find that post-LN encoders do not suffer from gradient imbalance. They thus conjecture that the gradient issue is not the direct cause of unstable post-LN Transformer training and further identify the *amplification effect* in post-LN Transformers — at initialization, the heavier dependency on residual branch leads to a larger output shift in post-LN Transformers, thus resulting in unstable training. In light of this finding, they introduce additional parameters to post-LN Transformers to control residual dependencies of Post-LN. These parameters are initialized according to activation variations of sample data so that the output shift of post-LN Transformers is not amplified. This approach ensures and boosts convergence of post-LN Transformers and reaches better performance than pre-LN Transformers.

5.2.2. Substitutes of layer normalization

Xu et al. (2019) empirically observe that the learnable parameters in the LN module do not work in most experiments, and even increase the risk of overfitting. They further conclude from controlled experiments that the forward normalization is not the reason why LN works for Transformer. From analysis and experiments, it is concluded that the derivatives of the mean and variance re-center and re-scale the gradients and play a significant role in LN. They thus propose *AdaNorm*, a normalization technique without learnable parameters

$$\mathbf{z} = C(1 - k\mathbf{y}) \odot \mathbf{y}, \tag{39}$$

$$\mathbf{y} = \frac{\mathbf{x} - \mu}{\sigma}, \tag{40}$$

where C, k are hyperparameters and $\odot$ denotes element-wise multiplication. μ and σ are the mean and standard deviation of input $\mathbf{x}$, respectively.

Nguyen and Salazar (2019) propose to replace the LN module with *scaled ℓ_2 normalization*. Given any input $\mathbf{x}$ of d-dimension, their approach project it onto a $d-1$-sphere of learned radius g

$$\mathbf{z} = g \frac{\mathbf{x}}{\|\mathbf{x}\|}, \tag{41}$$

where g is a learnable scalar. It is more parameter efficient compared to normal LN and is shown to be effective in machine translation datasets, especially in low-resource settings.

Shen et al. (2020) discuss why Batch Normalization (BN) (Ioffe and Szegedy, 2015) performs poorly in Transformer for text data and

[10] To the best of our knowledge, this approach is adopted since `v1.1.7` in the `Tensor2Tensor` implementation (Vaswani et al., 2018).

[11] Learning rate warm-up refers to starting optimization with an extremely small learning rate and then gradually increasing it to a pre-defined maximum value in a certain number of iterations.

conclude that BN's significant performance degradation stems from the instabilities associated with its batch statistics. They thus propose PowerNorm (PN) that has three modifications over BN: (1) it relaxes the zero-mean normalization; (2) it uses the quadratic mean of the signal, instead of the variance; (3) it uses running statistics for the quadratic mean, instead of using per-batch statistics. Specifically, for the tth iteration, the PN computes the outputs as

$$\mathbf{z}^{(t)} = \gamma \odot \mathbf{y}^{(t)} + \beta, \tag{42}$$

$$\mathbf{y}^{(t)} = \frac{\mathbf{x}^{(t)}}{\psi^{(t-1)}}, \tag{43}$$

$$(\psi^{(t)})^2 = \alpha(\psi^{(t-1)})^2 + (1-\alpha)\left(\frac{1}{|B|}\sum_{i=1}^{|B|}(\mathbf{x}_i^{(t)})^2\right), \tag{44}$$

where $0 < \alpha < 1$ is the moving average coefficient and γ, β are the learnable parameters as in BN formulation.

5.2.3. Normalization-free Transformer

Besides LN, there is another mechanism to construct deeper neural network. ReZero (Bachlechner et al., 2020) replace LN module with a learnable residual connection. For each module $F(\cdot)$, ReZero re-scales $F(\cdot)$ in the residual formulation:

$$\mathbf{H}' = \mathbf{H} + \alpha \cdot F(\mathbf{H}), \tag{45}$$

where α is a learnable parameter with zero-initialization.

Replacing LN in Transformer with ReZero mechanism is verified to induce better dynamic isometry for input signals and leads to faster convergence.

5.3. Position-wise FFN

Despite its simplicity, the position-wise feed-forward network (FFN) layers are important for a Transformer to achieve good performance. Dong et al. (2021) observe that simply stacking self-attention modules causes a *rank collapse* problem, leading to token-uniformity inductive bias, and that the feed-forward layer is one of the important building blocks that mitigate this issue. Various works have explored modifications on the FFN module.

5.3.1. Activation function in FFN

The vanilla Transformer (Vaswani et al., 2017) adopts the Rectified Linear Units (ReLU) activation for non-linearity in between the two FFN layers. Over time, several studies have explored different activation other than ReLU.

Ramachandran et al. (2018) try to replace ReLU in Transformer with Swish function $f(x) = x\text{sigmoid}(\beta x)$ and observe that it consistently improve performance on WMT 2014 English→German dataset.

GPT (Radford et al., 2018) replace ReLU with Gaussian Error Linear Unit (GELU) (Hendrycks and Gimpel, 2020) on language pre-training. It becomes the default practice for many pre-trained language models (Devlin et al., 2019; He et al., 2020a).

Shazeer (2020) explore using Gated Linear Units (GLU) (Dauphin et al., 2017) and its variants as a drop-in replacement for ReLU in FFN. Their pre-training experiments show that the GLU variants consistently improve vanilla Transformer with ReLU activation. Note that GLU introduces extra parameters and the experiments are conducted with the intermediate dimension of FFN reduced to match the parameter count with baseline.

5.3.2. Adapting FFN for larger capacity

Several works have focused on expanding FFNs in order for a larger model capacity. The basic idea is to replace FFNs with similar structures with much more parameters.

Lample et al. (2019) replace some of the FFNs with the product-key memory layers. A product-key memory is composed of three components: a query network, a key selection module containing two sets of sub-keys, and a value lookup table. The model first projects an input to a latent space using the query network, and then compares the generated query to keys that are Cartesian product of the two sets of sub-keys from key selection module to get k nearest neighbors, and finally finds the corresponding values in a value lookup table using the k nearest keys and aggregates them to produce the final output. This process resembles the attention mechanism, in that the generated query attends to a large number of global key–value pairs. They thus propose a multi-head mechanism for the key-product memory to further enlarge the capacity of this module. The experiments on large-scale language modeling suggest that this mechanism significantly improves performance with negligible computational overhead.

Several studies exploits the idea of Mixture-of-Experts (MoE) (Shazeer et al., 2017) to increase the capacity of FFNs. Gshard (Lepikhin et al., 2020) uses sparsely-gated MoE layers to replace FFNs in Transformer. Each MoE layer consists of several FFNs (each called an expert) that are the same structure as position-wise FFNs in vanilla Transformer. The output of the layer is a weighted sum of the outputs of the FFNs, using gate values computed by a routing function $g(\cdot)$. They design a learnable routing function that assigns tokens to experts, with auxiliary loss to satisfy balanced loads between experts and efficiency at the scale of length such that the experts can be distributed across multiple devices. For each forward pass of the MoE layer, only the experts with top-k gate values are activated.

Instead of using k experts for each forward pass, Switch Transformer (Fedus et al., 2022) proposes to route using only a single expert with the largest gate value, leading to a much smaller computational footprint. The authors also design an auxiliary loss to encourage load balance between experts. It is reported to speed up pre-training by a large margin compared to the non-MoE counterpart while having a similar number of FLOPS.

Yang et al. (2021) propose to replace top-k routing with expert prototyping strategy. Specifically, the proposed strategy splits experts into k different groups and applies top-1 routing within each group. The outputs of prototype groups are combined linearly to form the final output of the MoE layer. This strategy is proved to improve the model quality while maintaining constant computational costs.

As opposed to using a learnable routing function for expert assignment, Roller et al. (2021) design *hash layers* where tokens are hashed into a fixed number of buckets, each bucket corresponding to an expert. This approach requires no routing parameters or any auxiliary loss function, while showing competitive results with existing methods such as Switch Transformer (Fedus et al., 2022).

5.3.3. Dropping FFN layers

Notably, one might argue that under some circumstances, FFN layers can be dropped completely, resulting in a simplified network.

Sukhbaatar et al. (2019b) demonstrate that replacing the ReLU activation with Softmax and dropping the bias term in FFN effectively turns FFN into an attention module where position-wise inputs attend to a global key–value memory of D_{ffn} slots. They thus propose to drop the FFN module and add to the attention module a set of global key–value pairs, which are learnable parameters concatenated with key and values generated by inputs. This approach simplifies the structure of the network with no loss of performance.

Yang et al. (2020) empirically show that FFNs in the decoder of Transformer, despite its large number of parameters, is not efficient and can be removed safely with only slight or no loss of performance. This approach significantly boosts the training and inference speed.

6. Architecture-level variants

In this section, we introduce the X-formers that modify the vanilla Transformer beyond modules.

6.1. Adapting Transformer to be lightweight

Apart from the efforts made at the module level to alleviate computation overheads, there are several attempts to adapt Transformer to be lightweight by modifications at a higher level.

Similar to low-rank self-attention (Guo et al., 2019b) that decomposes attention into a locality-constrained attention and a low-rank global attention, Lite Transformer (Wu et al., 2020b) proposes to replace each attention module in Transformer with a two-branch structure, where one branch uses attention to capture long-range contexts while the other branch uses depth-wise convolution and linear layers to capture local dependencies. The architecture is lightweight both in terms of model size and computation, and is thus more suitable for mobile devices.

Funnel Transformer (Dai et al., 2020) utilizes a funnel-like encoder architecture where the length of the hidden sequence is gradually reduced using pooling along the sequence dimension, and then recovered using up-sampling. The architecture effectively reduces the FLOPs and memory compared to the vanilla Transformer encoder. Naturally, one can use this architecture to build a deeper or wider model using the same computation resources.

DeLighT (Mehta et al., 2020) replaces the standard Transformer block with `DeLighT` block, which consists of three sub-modules: (1) a "expand-and-reduce" `DeLighT` transformation module to learn wider representations with low computation requirements; (2) a single-head self-attention to learn pair-wise interaction; (3) a lightweight "reduce-and-expand" FFN (as opposed to vanilla Transformer that first expands the dimension of hidden representations and then reduces them back to D_m). They also propose a block-wise scaling strategy that allows for shallower and narrower blocks near the input and wider and deeper blocks near the output. The induced network is much deeper than the vanilla Transformer but with fewer parameters and operations.

6.2. Strengthening cross-block connectivity

In vanilla Transformer, each block takes outputs from the previous block as inputs and outputs a sequence of hidden representations. One might be interested in creating more paths along which input signals can run through the networks. In Section 4.5.3, we introduced Realformer (He et al., 2020b) and Predictive Attention Transformer (Wang et al., 2021b) that reuses attention distributions from previous block to guide attention of current block. This can be seen as creating a forward path between adjacent Transformer blocks.

In a deep Transformer encoder–decoder model, the cross-attention modules in the decoder only utilize the final outputs of the encoder, therefore the error signal will have to traverse along the depth of the encoder. This makes Transformer more susceptible to optimization issues (e.g., vanishing gradients). Transparent Attention (Bapna et al., 2018) uses a weighted sum of encoder representations at all encoder layers (including the embedding layer) in each cross-attention module. For the jth decoder block, the cross-attention module is modified to attend to

$$\tilde{\mathbf{H}}^{(j)} = \sum_{i=0}^{N} \frac{\exp(w_{ij})}{\sum_{k=0}^{N} \exp(w_{kj})} \mathbf{H}^{(i)}, \tag{46}$$

where each w_{ij} is a trainable parameter. This effectively shortens the path from each layer in the encoder to the error signal and thus eases the optimization of deeper Transformer models.

Another issue associated with vanilla Transformer is that each position can only attend to history representations from lower layers. Feedback Transformer (Fan et al., 2021b) proposes to add a feedback mechanism to Transformer decoder, where each position attends to a weighted sum of history representations from all layers

$$\tilde{\mathbf{h}}_i = \sum_{l=0}^{N} \frac{\exp(w_l)}{\sum_{k=0}^{N} \exp(w_k)} \mathbf{h}_i^{(l)}. \tag{47}$$

6.3. Adaptive computation time

Vanilla Transformer, like most neural models, utilizes a fixed (learned) computation procedure to process each input. An intriguing and promising modification is to make computation time conditioned on the inputs, i.e., to introduce Adaptive Computation Time (ACT) (Graves, 2016) into Transformer models. Such modifications potentially give rise to the following advantages:

- Feature refinement for hard examples. For data that are hard to process, a shallow representation might not be adequate to fulfill the task at hand. It would be more ideal to apply more computations to acquire a deeper and more refined representation.
- Efficiency for easy examples. When processing easy examples, a shallow representation might be enough for the task. In this case, it would be beneficial if the network can learn to extract features using reduced computation time.

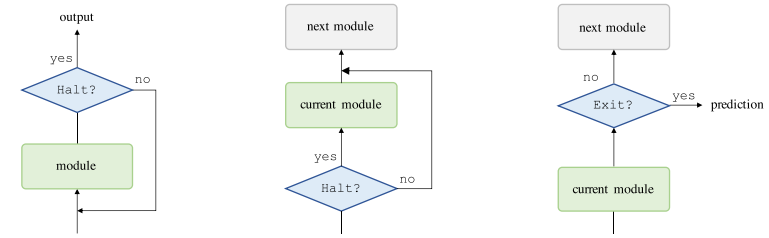
Universal Transformer (UT) (Dehghani et al., 2019) incorporates a recurrence-over-depth mechanism that iteratively refines representations for all symbols using a module that is shared over depth, as illustrated in Fig. 11(a). It also adds a per-position dynamic halting mechanism that calculates a halting probability for each symbol at every time step. If a symbol's halting probability is greater than a predefined threshold, then the symbol's representation will remain unchanged for subsequent timesteps. The recurrence is stopped when all symbols halt or when a predefined maximum step is reached.

Conditional Computation Transformer (CCT) (Bapna et al., 2020) adds a gating module at each self-attention and feed-forward layer to decide whether to skip the current layer, as illustrated in Fig. 11(b). The authors also introduce an auxiliary loss that encourages the model to adjust the gating modules to match the practical computation cost to the available computation budget.

Similar to the dynamic halting mechanism used in UT, there is a line of work dedicated to adapting the number of layers to each input in order to achieve a good speed-accuracy trade-off, which is called *early exit* mechanism, as illustrated in Fig. 11(c). A commonly used technique is to add an internal classifier at each layer and jointly train all classifiers. The core of these methods is the criteria used to decide whether to exit at each layer. DeeBERT (Xin et al., 2020) uses the entropy of the output probability distribution of the current layer to determine whether to exit. PABEE (Zhou et al., 2020) counts the number of times that the predictions remain unchanged to decide whether to exit. Li et al. (2021) design a window-based uncertainty criterion to achieve token-level partial exiting for sequence labeling tasks. Sun et al. (2021) introduces a voting-based exiting strategy that considers at each layer predictions of all the past internal classifiers to infer the correct label and to decide whether to exit.

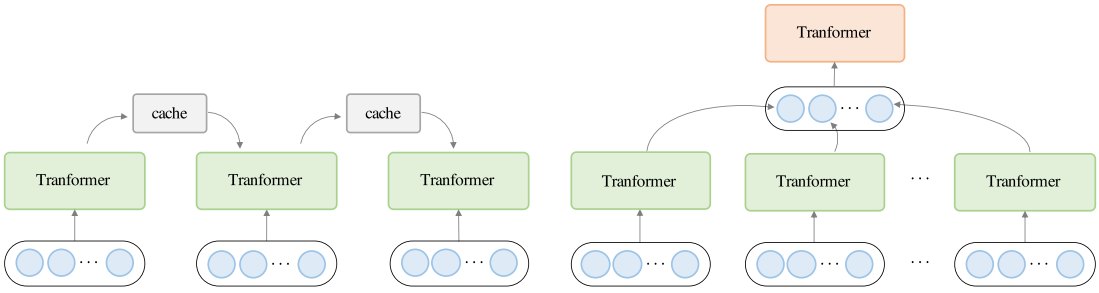
6.4. Transformers with divide-and-conquer strategies

The quadratic complexity of self-attention on sequences length can significantly limit the performance of some downstream tasks. For example, language modeling usually needs long-range context. Apart from the techniques introduced in Section 4, another effective way of dealing with long sequences is to use *divide-and-conquer* strategy, i.e., to decompose an input sequence into finer segments that can be efficiently processed by Transformer or Transformer modules. We identify two representative class of methods, *recurrent* and *hierarchical* Transformers, as illustrated in Fig. 12. These techniques can be understood as a wrapper for the Transformer model in which Transformer acts as an elementary component that is reused to process different input segments.



(a) dynamic halting (b) conditional skipping (c) early exit

Fig. 11. Three typical ACT paradigms.



(a) Recurrent Transformer (b) Hierarchical Transformer

Fig. 12. Illustrations of recurrent and hierarchical Transformers.

6.4.1. Recurrent Transformers

In recurrent Transformers, a cache memory is maintained to incorporate the history information. While processing a segment of text, the network reads from the cache as an additional input. After the processing is done, the network writes to the memory by simply copying hidden states or using more complex mechanisms. The abstract process is illustrated in Fig. 12(a).

Transformer-XL (Dai et al., 2019) address the limitation of a fixed length context by caching representations from the previous segment and reuse it as an extended context when the model processes the current segment. For the lth layer and the $(\tau+1)$-th segment, the input representation $\mathbf{H}_{\tau+1}^{(l-1)}$ is concatenated with the representation $\mathbf{H}_{\tau}^{(l-1)}$ from previous segment to produce the keys and values

$$\tilde{\mathbf{H}}_{\tau+1}^{(l)} = [\mathrm{SG}(\mathbf{H}_{\tau}^{(l-1)}) \circ \mathbf{H}_{\tau+1}^{(l-1)}], \tag{48}$$

$$\mathbf{K}_{\tau+1}^{(l)}, \mathbf{V}_{\tau+1}^{(l)} = \tilde{\mathbf{H}}_{\tau+1}^{(l)} \mathbf{W}^{K}, \tilde{\mathbf{H}}_{\tau+1}^{(l)} \mathbf{W}^{V}, \tag{49}$$

where $\mathbf{H}_{\tau}^{(0)}$ is defined as the word embedding sequence, $\mathrm{SG}(\cdot)$ denotes stop-gradient operation and $[\mathbf{X} \circ \mathbf{Y}]$ denotes concatenating the two vector sequences along the time dimension. This approach extends the maximum context length by $L \times N_{\mathrm{mem}}$ where L is the number of layers and N_{mem} is the length of cached memory sequence.

Compressive Transformer (Rae et al., 2020) extends this idea further by extending the cache with two levels of memory. In Transformer-XL, the activations from the previous segment are cached as a memory that is used to augment the current segment, and activations from older segments are discarded. Compressive Transformer, on the other hand, applies a compression operation (e.g., Convolution, Pooling, etc.) on older activations and stores them in the compressed memory. In order to avoid the expensive backpropagating-through-time (BPTT) from training compression sub-network with gradients from the loss, they propose to use local loss functions where original memories are constructed from the compressed memories. This approach further extends the theoretical maximum history context length from $L \times N_{\mathrm{mem}}$ of Transformer-XL to $L \times (N_{\mathrm{mem}} + c \times N_{\mathrm{cm}})$, where c is the compression rate and N_{cm} is the length of compressed memory.

Memformer (Wu et al., 2020a) extends the recurrence mechanism from decoder-only architecture to an encoder–decoder architecture. They introduce to the encoder a memory cross attention similar to the cross attention in vanilla Transformer to allow the Transformer encoder to attend to the memory. They also introduce a memory slot attention on top of the encoder output to explicitly write the memory for the next segment. To avoid BPTT over a long range of timesteps, they propose Memory Replay Back-Propagation (MRBP) algorithm, which replays the memory at each timestep to accomplish gradient back-propagation over long unrolls.

Yoshida et al. (2020) propose a simple fine-tuning mechanism to add recurrence to a pre-trained language model (e.g., GPT-2 (Radford et al., 2019)).

ERNIE-Doc (Ding et al., 2020) proposes an enhanced recurrence mechanism based on the recurrence mechanism used in Transformer-XL, by replacing the memory with the history representations from the lth layer.

6.4.2. Hierarchical Transformers

Hierarchical Transformer decomposes inputs hierarchically into elements of finer granularity. Low-level features are first fed to a Transformer encoder, producing output representations that are then aggregated (using pooling or other operations) to form a high-level feature, which is then processed by a high-level Transformer. This class of methods can be understood as a process of hierarchical abstraction. The overview of this approach is depicted in Fig. 12(b). The advantages of this approach are twofold: (1) Hierarchical modeling allows the model to handle long inputs with limited resources; (2) It has the potential to generate richer representations that are beneficial to tasks.

6.4.2.1. Hierarchical for long sequence inputs. For tasks with inherently long input length, one can use hierarchical Transformers for effective modeling of long-range dependencies. For document-level machine translation tasks, Miculicich et al. (2018) introduce dependencies on the previous sentences from both the source and target sides when translating a sentence. They use an attention mechanism as the aggregation operation to summarize low-level information. For document summarization, HIBERT (Zhang et al., 2019) encodes a document of text by first learn sentence representations for all sentences and then use these sentence representations to encode document-level representations that are then used to generate the summary. The model uses the last hidden representation (corresponding to the EOS token) as

the representation for each sentence. Liu and Lapata (2019) propose a similar hierarchical Transformer for multi-document summarization where the extracted low-level representations are aggregated using an attention layer with a global trainable query node and low-level representations as the source of key–value pairs. Hi-Transformer (Wu et al., 2021b) first utilizes a sentence Transformer and a document Transformer to hierarchically learn document context-aware sentence representations. The document context-aware sentence representations are then fed to another sentence Transformer to further improve the sentence context modeling.

6.4.2.2. Hierarchical for richer representations. One might also be interested in using hierarchical models to acquire richer representations that are beneficial to the tasks at hand. For example, TENER (Yan et al., 2019) uses a low-level Transformer encoder to encode character features, which is then concatenated with word embeddings as the inputs to the high-level Transformer encoder. This incorporates more features and alleviates the problems of data sparsity and out-of-vocabulary (OOV). Recently emerging Vision Transformer (Dosovitskiy et al., 2020) divides an input image into several patches that serve as the basic input elements of Transformer, which potentially loses intrinsic pixel-level information within patches. To address this issue, Transformer in Transformer (TNT) (Han et al., 2021c) uses at each layer an inner Transformer block that transforms pixel representations and an outer Transformer block that takes fused vectors of patch representations and pixel representations as input.

6.5. Exploring alternative architecture

Despite the success of Transformer architecture, one might question whether the current Transformer architecture is optimal. Interestingly, several studies have explored alternative architectures for Transformer.

Lu et al. (2020) interpret Transformer as a numerical Ordinary Differential Equation (ODE) solver for a convection–diffusion equation in a multi-particle dynamic system and design Macaron Transformer, which replaces each Transformer block with a *FFN-attention-FFN* variant.

Sandwich Transformer (Press et al., 2020) explores reorganizing attention modules and FFN modules such that attention modules are mainly located in lower layers and FFN modules in upper layers. The induced model improves perplexity on multiple language modeling benchmarks, without increasing parameters, memory or training time.

Mask Attention Network (MAN) (Fan et al., 2021a) prepends a dynamic mask attention module to the self-attention module in each Transformer block. The mask is conditioned on token representations, the relative distance between tokens and head indices. The proposed dynamic mask attention is shown to effectively model locality in text data and the induced model consistently outperforms the baseline model in machine translation and abstractive summarization.

Notably, there is a line of work that uses Neural Architecture Search (NAS) to search for alternative Transformer architectures. The Evolved Transformer (ET) (So et al., 2019) employs evolution-based architecture search with the standard Transformer architecture seeding the initial population. The searched model demonstrates consistent improvement over Transformer on several language tasks. As another representative work, DARTSformer (Zhao et al., 2021) applies differentiable architecture search (DARTS) (Liu et al., 2019b), combined with a multi-split reversible network and a backpropagation-with-reconstruction algorithm for memory efficiency. The resulting model consistently outperforms standard Transformer and compares favorably to larger ET models, with a significantly reduced search cost.

7. Pre-trained Transformers

As a key difference from convolutional networks and recurrent networks that inherently incorporates the inductive bias of locality, Transformer does not make any assumption about how the data is structured. On the one hand, this effectively makes Transformer a very universal architecture that has the potential of capturing dependencies of different ranges. On the other hand, this makes Transformer prone to overfitting when the data is limited. One way to alleviate this issue is to introduce inductive bias into the model.

Recent studies suggest that Transformer models that are pre-trained on large corpora can learn universal language representations that are beneficial for downstream tasks (Qiu et al., 2020). The models are pre-trained using various self-supervised objectives, e.g., predicting a masked word given its context. After pre-training a model, one can simply fine-tune it on downstream datasets, instead of training a model from scratch. To illustrate typical ways of using Transformers in pre-training, we identify some of the pre-trained Transformers and categorize them as follows.

- *Encoder only*. A line of work uses the Transformer encoder as its backbone architecture. BERT (Devlin et al., 2019) is a representative PTM that is typically used for natural language understanding tasks. It utilizes Masked Language Modeling (MLM) and Next Sentence Prediction (NSP) as the self-supervised training objective. RoBERTa (Liu et al., 2019a) further adapts the training of BERT and removes the NSP objective as it is found to hurt performance on downstream tasks.
- *Decoder only*. Several studies focus on pre-training Transformer decoders on language modeling. For example, the Generative Pre-trained Transformer (GPT) series (i.e., GPT (Radford et al., 2018), GPT-2 (Radford et al., 2019), and GPT-3 (Brown et al., 2020)) is dedicated to scaling pre-trained Transformer decoders and has recently illustrated that a large-scale PTM can achieve impressive few-shot performance with the task and examples fed to the model as constructed prompts (Brown et al., 2020).
- *Encoder–Decoder*. There are also PTMs that adopt Transformer encoder–decoder as the overall architecture. BART (Lewis et al., 2020) extends the denoising objective of BERT to encoder–decoder architecture. The benefit of using an encoder–decoder architecture is that the inducing model is equipped with the ability to perform both natural language understanding and generation. T5 (Raffel et al., 2020) adopts similar architecture and was one of the earliest studies that use task-specific text prefix in downstream tasks.

Some of the Transformer architecture variants can also be applied to Transformer-based PTMs. For instance, BigBird (Zaheer et al., 2020) introduced in Section 4.1 is a encoder-based PTM that uses compound position-based sparse attention to enable long sequence inputs. GPT-3 (Brown et al., 2020) uses alternating dense and locally banded sparse attention (which was also introduced in Section 4.1) in self-attention modules. Switch Transformer (Fedus et al., 2022) is an encoder-based PTM that replaces FFN layers with mixture-of-experts layers and can increase parameter count while keeping the FLOPs per example constant.

8. Applications of Transformer

Transformer was originally designed for machine translation but has been widely adopted in various fields besides NLP, including CV and audio processing, due to its flexible architecture.

(1) *Natural Language Processing*. Transformer and its variants have been extensively explored and applied in NLP tasks, e.g., machine translation (Vaswani et al., 2017; Mehta et al., 2020; Raffel et al., 2020; So et al., 2019; Fan et al., 2021a), language modeling (Dai et al., 2019; Shoeybi et al., 2020; Roy et al., 2021; Rae et al., 2020) and named entity recognition (Yan et al., 2019; Li et al., 2020c). Massive effort has been dedicated to pre-training Transformer models on large-scale text corpora, which we believe is one of the major reasons of Transformer's wide application in NLP.

(2) *Computer Vision*. Transformer have also been adapted for various vision tasks, e.g., image classification (Chen et al., 2020b; Dosovitskiy et al., 2020; Liu et al., 2021), object detection (Carion et al., 2020; Zhu et al., 2020; Zheng et al., 2020a; Liu et al., 2021), image generation (Parmar et al., 2018; Jiang et al., 2021) and video processing (Shao et al., 2021; Arnab et al., 2021). Han et al. (2021a) and Khan et al. (2021) provide reviews on existing work of visual Transformers. We encourage readers to refer to these surveys for further understand the current research progress on Transformers in CV.

(3) *Audio Applications*. Transformer can also be extended for audio-related applications, e.g., speech recognition (Dong et al., 2018; Pham et al., 2019; Chen et al., 2021; Gulati et al., 2020), speech synthesis (Li et al., 2019a; Zheng et al., 2020b; Ihm et al., 2020), speech enhancement (Kim et al., 2020; Yu et al., 2021) and music generation (Huang et al., 2019).

(4) *Multimodal Applications*. Owing to its flexible architecture, Transformer has also been applied in various multimodal scenarios, e.g., visual question answering (Li et al., 2019c; Hu et al., 2020; Su et al., 2020; Li et al., 2020a), visual commonsense reasoning (Li et al., 2019c; Su et al., 2020), caption generation (Sun et al., 2019; Cornia et al., 2020; Lin et al., 2021), speech-to-text translation (Han et al., 2021b) and text-to-image generation (Ramesh et al., 2021; Lin et al., 2021; Ding et al., 2021).

9. Conclusion and future directions

In this survey, we conduct a comprehensive overview of X-formers and propose a new taxonomy. Most of the existing works improve Transformer from different perspectives, such as efficiency, generalization, and applications. The improvements include incorporating structural prior, designing lightweight architecture, pre-training, and so on.

Although X-formers have proven their power for various tasks, challenges still exist. Besides the current concerns (e.g. efficiency and generalization), the further improvements of Transformer may lie in the following directions:

(1) *Theoretical Analysis*. The architecture of Transformer has been demonstrated to be capable of supporting large-scale training datasets with enough parameters. Many works show that Transformer has a larger capacity than CNNs and RNNs and hence has the ability to handle a huge amount of training data. When Transformer is trained on sufficient data, it usually has better performances than CNNs or RNNs. An intuitive explanation is that Transformer has few prior assumptions on the data structure and therefore is more flexible than CNNs and RNNs. However, the theoretical reason is unclear and we need some theoretical analysis of Transformer ability.

(2) *Better Global Interaction Mechanism beyond Attention*. A main advantage of Transformer is the use of the attention mechanism to model the global dependencies among nodes within input data. However, many studies have shown that full attention is unnecessary for most nodes. It is, to some degree, inefficient to indistinguishably calculate attention for all nodes. Therefore, there is still plenty of room for improvements in efficiently modeling global interactions. On the one hand, the self-attention module can be regarded as a fully-connected neural network with dynamical connection weights, which aggregates non-local information with dynamic routing. Therefore, other dynamic routing mechanisms are alternative approaches worth exploring. On the other hand, the global interaction can also be modeled by other types of neural networks, such as memory-enhanced models.

(3) *Unified Framework for Multimodal Data*. In many application scenarios, integrating multimodal data is useful and necessary to boost the task performance. Moreover, the general AI also needs the ability to capture the semantic relations across different modalities. Since Transformer achieves great success on text, image, video, and audio, we have a chance to build a unified framework and better capture the inherent connections among multimodal data. However, the design of the intra-modal and cross-modal attention still remains to be improved.

Finally, we wish this survey to be a hands-on reference for better understanding the current research progress on Transformers and help readers to further improve Transformers for various applications.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by the National Natural Science Foundation of China (No.62022027).

References

Ainslie, Joshua, Ontanon, Santiago, Alberti, Chris, Cvicek, Vaclav, Fisher, Zachary, Pham, Philip, Ravula, Anirudh, Sanghai, Sumit, Wang, Qifan, Yang, Li, 2020. ETC: Encoding long and structured inputs in transformers. In: Proceedings of EMNLP. pp. 268–284. http://dx.doi.org/10.18653/v1/2020.emnlp-main.19, Online.

Al-Rfou, Rami, Choe, Dokook, Constant, Noah, Guo, Mandy, Jones, Llion, 2019. Character-level language modeling with deeper self-attention. In: Proceedings of AAAI. pp. 3159–3166. http://dx.doi.org/10.1609/aaai.v33i01.33013159.

Arnab, Anurag, Dehghani, Mostafa, Heigold, Georg, Sun, Chen, Lučić, Mario, Schmid, Cordelia, 2021. Vivit: A video vision transformer. arXiv:2103.15691.

Ba, Lei Jimmy, Kiros, Jamie Ryan, Hinton, Geoffrey E., 2016. Layer normalization. CoRR arXiv:1607.06450.

Bachlechner, Thomas, Majumder, Bodhisattwa Prasad, Mao, Huanru Henry, Cottrell, Garrison W., McAuley, Julian J., 2020. Rezero is all you need: Fast convergence at large depth. CoRR arXiv:2003.04887.

Baevski, Alexei, Auli, Michael, 2019. Adaptive input representations for neural language modeling. In: Proceedings of ICLR. URL https://openreview.net/forum?id=ByxZX20qFQ.

Bapna, Ankur, Arivazhagan, Naveen, Firat, Orhan, 2020. Controlling computation versus quality for neural sequence models. arXiv:2002.07106, [cs.LG].

Bapna, Ankur, Chen, Mia, Firat, Orhan, Cao, Yuan, Wu, Yonghui, 2018. Training deeper neural machine translation models with transparent attention. In: Proceedings of EMNLP. Brussels, Belgium, pp. 3028–3033. http://dx.doi.org/10.18653/v1/D18-1338.

Battaglia, Peter W., Hamrick, Jessica B., Bapst, Victor, Sanchez-Gonzalez, Alvaro, Zambaldi, Vinicius, Malinowski, Mateusz, Tacchetti, Andrea, Raposo, David, Santoro, Adam, Faulkner, Ryan, Gulcehre, Caglar, Song, Francis, Ballard, Andrew, Gilmer, Justin, Dahl, George, Vaswani, Ashish, Allen, Kelsey, Nash, Charles, Langston, Victoria, Dyer, Chris, Heess, Nicolas, Wierstra, Daan, Kohli, Pushmeet, Botvinick, Matt, Vinyals, Oriol, Li, Yujia, Pascanu, Razvan, 2018. Relational inductive biases, deep learning, and graph networks. arXiv:1806.01261, [cs.LG].

Beltagy, Iz, Peters, Matthew E., Cohan, Arman, 2020. Longformer: The long-document transformer. arXiv:2004.05150, [cs.CL].

Bhojanapalli, Srinadh, Yun, Chulhee, Rawat, Ankit Singh, Reddi, Sashank J., Kumar, Sanjiv, 2020. Low-rank bottleneck in multi-head attention models. In: Proceedings of ICML. pp. 864–873, URL http://proceedings.mlr.press/v119/bhojanapalli20a.html.

Brown, Tom, Mann, Benjamin, Ryder, Nick, Subbiah, Melanie, Kaplan, Jared D, Dhariwal, Prafulla, Neelakantan, Arvind, Shyam, Pranav, Sastry, Girish, Askell, Amanda, Agarwal, Sandhini, Herbert-Voss, Ariel, Krueger, Gretchen, Henighan, Tom, Child, Rewon, Ramesh, Aditya, Ziegler, Daniel, Wu, Jeffrey, Winter, Clemens, Hesse, Chris, Chen, Mark, Sigler, Eric, Litwin, Mateusz, Gray, Scott, Chess, Benjamin, Clark, Jack, Berner, Christopher, McCandlish, Sam, Radford, Alec, Sutskever, Ilya, Amodei, Dario, 2020. Language models are few-shot learners. In: Proceedings of NeurIPS. pp. 1877–1901, URL https://proceedings.neurips.cc/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf.

Carion, Nicolas, Massa, Francisco, Synnaeve, Gabriel, Usunier, Nicolas, Kirillov, Alexander, Zagoruyko, Sergey, 2020. End-to-end object detection with transformers. In: Proceedings of ECCV. pp. 213–229. http://dx.doi.org/10.1007/978-3-030-58452-8_13.

Chen, Ziye, Gong, Mingming, Ge, Lingjuan, Du, Bo, 2020a. Compressed self-attention for deep metric learning with low-rank approximation. In: Proceedings of IJCAI. pp. 2058–2064. http://dx.doi.org/10.24963/ijcai.2020/285.

Chen, Mark, Radford, Alec, Child, Rewon, Wu, Jeffrey, Jun, Heewoo, Luan, David, Sutskever, Ilya, 2020b. Generative pretraining from pixels. In: Proceedings of ICML. pp. 1691–1703, URL http://proceedings.mlr.press/v119/chen20s.html.

Chen, Xie, Wu, Yu, Wang, Zhenghao, Liu, Shujie, Li, Jinyu, 2021. Developing real-time streaming transformer transducer for speech recognition on large-scale dataset. arXiv:2010.11395, [cs.CL].

Child, Rewon, Gray, Scott, Radford, Alec, Sutskever, Ilya, 2019. Generating long sequences with sparse transformers. arXiv:1904.10509, [cs.LG].

Choromanski, Krzysztof, Likhosherstov, Valerii, Dohan, David, Song, Xingyou, Gane, Andreea, Sarlos, Tamas, Hawkins, Peter, Davis, Jared, Belanger, David, Colwell, Lucy, Weller, Adrian, 2020a. Masked language modeling for proteins via linearly scalable long-context transformers. arXiv:2006.03555, [cs.LG].

Choromanski, Krzysztof, Likhosherstov, Valerii, Dohan, David, Song, Xingyou, Gane, Andreea, Sarlos, Tamas, Hawkins, Peter, Davis, Jared, Mohiuddin, Afroz, Kaiser, Lukasz, Belanger, David, Colwell, Lucy, Weller, Adrian, 2020b. Rethinking attention with performers. arXiv:2009.14794, [cs.LG].

Chu, Xiangxiang, Tian, Zhi, Zhang, Bo, Wang, Xinlong, Wei, Xiaolin, Xia, Huaxia, Shen, Chunhua, 2021. Conditional positional encodings for vision transformers. arXiv:2102.10882, [cs.CV].

Cordonnier, Jean-Baptiste, Loukas, Andreas, Jaggi, Martin, 2020. Multi-head attention: Collaborate instead of concatenate. CoRR arXiv:2006.16362.

Cornia, Marcella, Stefanini, Matteo, Baraldi, Lorenzo, Cucchiara, Rita, 2020. Meshed-memory transformer for image captioning. In: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2020, Seattle, WA, USA, June 13-19, 2020. IEEE, pp. 10575–10584. http://dx.doi.org/10.1109/CVPR42600.2020.01059.

Dai, Zihang, Lai, Guokun, Yang, Yiming, Le, Quoc, 2020. Funnel-transformer: Filtering out sequential redundancy for efficient language processing. In: Proceedings of NeurIPS. URL https://proceedings.neurips.cc/paper/2020/hash/2cd2915e69546904e4e5d4a2ac9e1652-Abstract.html.

Dai, Zihang, Yang, Zhilin, Yang, Yiming, Carbonell, Jaime, Le, Quoc, Salakhutdinov, Ruslan, 2019. Transformer-XL: Attentive language models beyond a fixed-length context. In: Proceedings of ACL. Florence, Italy, pp. 2978–2988. http://dx.doi.org/10.18653/v1/P19-1285.

Dauphin, Yann N., Fan, Angela, Auli, Michael, Grangier, David, 2017. Language modeling with gated convolutional networks. In: Proceedings of ICML. pp. 933–941, URL http://proceedings.mlr.press/v70/dauphin17a.html.

Dehghani, Mostafa, Gouws, Stephan, Vinyals, Oriol, Uszkoreit, Jakob, Kaiser, Lukasz, 2019. Universal transformers. In: Proceedings of ICLR. URL https://openreview.net/forum?id=HyzdRiR9Y7.

Deshpande, Ameet, Narasimhan, Karthik, 2020. Guiding attention for self-supervised learning with transformers. In: Findings of the Association for Computational Linguistics: EMNLP 2020. pp. 4676–4686. http://dx.doi.org/10.18653/v1/2020.findings-emnlp.419, Online.

Devlin, Jacob, Chang, Ming-Wei, Lee, Kenton, Toutanova, Kristina, 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In: Proceedings of HLT-NAACL. Minneapolis, Minnesota, pp. 4171–4186. http://dx.doi.org/10.18653/v1/N19-1423.

Ding, Siyu, Shang, Junyuan, Wang, Shuohuan, Sun, Yu, Tian, Hao, Wu, Hua, Wang, Haifeng, 2020. ERNIE-DOC: The retrospective long-document modeling transformer. arXiv:2012.15688, [cs.CL].

Ding, Ming, Yang, Zhuoyi, Hong, Wenyi, Zheng, Wendi, Zhou, Chang, Yin, Da, Lin, Junyang, Zou, Xu, Shao, Zhou, Yang, Hongxia, Tang, Jie, 2021. CogView: Mastering text-to-image generation via transformers. arXiv:2105.13290, [cs.CV].

Dong, Yihe, Cordonnier, Jean-Baptiste, Loukas, Andreas, 2021. Attention is not all you need: Pure attention loses rank doubly exponentially with depth. CoRR arXiv:2103.03404.

Dong, Linhao, Xu, Shuang, Xu, Bo, 2018. Speech-transformer: A no-recurrence sequence-to-sequence model for speech recognition. In: Proceedings of ICASSP. pp. 5884–5888. http://dx.doi.org/10.1109/ICASSP.2018.8462506.

Dosovitskiy, Alexey, Beyer, Lucas, Kolesnikov, Alexander, Weissenborn, Dirk, Zhai, Xiaohua, Unterthiner, Thomas, Dehghani, Mostafa, Minderer, Matthias, Heigold, Georg, Gelly, Sylvain, Uszkoreit, Jakob, Houlsby, Neil, 2020. An image is worth 16x16 words: Transformers for image recognition at scale. arXiv:2010.11929, [cs.CV].

Fan, Zhihao, Gong, Yeyun, Liu, Dayiheng, Wei, Zhongyu, Wang, Siyuan, Jiao, Jian, Duan, Nan, Zhang, Ruofei, Huang, Xuanjing, 2021a. Mask attention networks: Rethinking and strengthen transformer. In: Proceedings of NAACL. pp. 1692–1701, URL https://www.aclweb.org/anthology/2021.naacl-main.135.

Fan, Angela, Lavril, Thibaut, Grave, Edouard, Joulin, Armand, Sukhbaatar, Sainbayar, 2021b. Addressing some limitations of transformers with feedback memory. URL https://openreview.net/forum?id=OCm0rwa1lx1.

Fedus, William, Zoph, Barret, Shazeer, Noam, 2022. Switch transformers: scaling to trillion parameter models with simple and efficient sparsity. Journal of Machine Learning Research 23 (120), 1–39, http://jmlr.org/papers/v23/21-0998.html.

Gehring, Jonas, Auli, Michael, Grangier, David, Yarats, Denis, Dauphin, Yann N., 2017. Convolutional sequence to sequence learning. In: Proceedings of ICML. pp. 1243–1252.

Graves, Alex, 2016. Adaptive computation time for recurrent neural networks. CoRR arXiv:1603.08983.

Gu, Shuhao, Feng, Yang, 2019. Improving multi-head attention with capsule networks. In: Proceedings of NLPCC. pp. 314–326. http://dx.doi.org/10.1007/978-3-030-32233-5_25.

Gu, Jiatao, Liu, Qi, Cho, Kyunghyun, 2019. Insertion-based decoding with automatically inferred generation order. Trans. Assoc. Comput. Linguist. 7, 661–676, URL https://transacl.org/ojs/index.php/tacl/article/view/1732.

Gulati, Anmol, Qin, James, Chiu, Chung-Cheng, Parmar, Niki, Zhang, Yu, Yu, Jiahui, Han, Wei, Wang, Shibo, Zhang, Zhengdong, Wu, Yonghui, Pang, Ruoming, 2020. Conformer: Convolution-augmented transformer for speech recognition. In: Proceedings of Interspeech. pp. 5036–5040. http://dx.doi.org/10.21437/Interspeech.2020-3015.

Guo, Qipeng, Qiu, Xipeng, Liu, Pengfei, Shao, Yunfan, Xue, Xiangyang, Zhang, Zheng, 2019a. Star-transformer. In: Proceedings of HLT-NAACL. pp. 1315–1325, URL https://www.aclweb.org/anthology/N19-1133.

Guo, Qipeng, Qiu, Xipeng, Liu, Pengfei, Xue, Xiangyang, Zhang, Zheng, 2020. Multi-scale self-attention for text classification. In: Proceedings of AAAI. pp. 7847–7854, URL https://aaai.org/ojs/index.php/AAAI/article/view/6290.

Guo, Qipeng, Qiu, Xipeng, Xue, Xiangyang, Zhang, Zheng, 2019b. Low-rank and locality constrained self-attention for sequence modeling. IEEE/ACM Trans. Audio Speech Lang. Proc. 27 (12), 2213–2222. http://dx.doi.org/10.1109/TASLP.2019.2944078.

Guo, Maosheng, Zhang, Yu, Liu, Ting, 2019c. Gaussian transformer: A lightweight approach for natural language inference. In: Proceedings of AAAI. pp. 6489–6496. http://dx.doi.org/10.1609/aaai.v33i01.33016489.

Han, Kai, Wang, Yunhe, Chen, Hanting, Chen, Xinghao, Guo, Jianyuan, Liu, Zhenhua, Tang, Yehui, Xiao, An, Xu, Chunjing, Xu, Yixing, Yang, Zhaohui, Zhang, Yiman, Tao, Dacheng, 2021a. A survey on visual transformer. arXiv:2012.12556, [cs.CV].

Han, Chi, Wang, Mingxuan, Ji, Heng, Li, Lei, 2021b. Learning shared semantic space for speech-to-text translation. arXiv:2105.03095, [cs.CL].

Han, Kai, Xiao, An, Wu, Enhua, Guo, Jianyuan, Xu, Chunjing, Wang, Yunhe, 2021c. Transformer in transformer. arXiv:2103.00112, [cs.CV].

He, Pengcheng, Liu, Xiaodong, Gao, Jianfeng, Chen, Weizhu, 2020a. Deberta: Decoding-enhanced BERT with disentangled attention. arXiv:2006.03654.

He, Ruining, Ravula, Anirudh, Kanagal, Bhargav, Ainslie, Joshua, 2020b. RealFormer: Transformer likes residual attention. arXiv:2012.11747, [cs.LG].

He, Kaiming, Zhang, Xiangyu, Ren, Shaoqing, Sun, Jian, 2016. Deep residual learning for image recognition. In: Proceedings CVPR. pp. 770–778. http://dx.doi.org/10.1109/CVPR.2016.90.

Hendrycks, Dan, Gimpel, Kevin, 2020. Gaussian error linear units (GELUs). arXiv:1606.08415, [cs.LG].

Hinton, Geoffrey E., Sabour, Sara, Frosst, Nicholas, 2018. Matrix capsules with EM routing. In: Proceedings of ICLR. URL https://openreview.net/forum?id=HJWLfGWRb.

Ho, Jonathan, Kalchbrenner, Nal, Weissenborn, Dirk, Salimans, Tim, 2019. Axial attention in multidimensional transformers. CoRR arXiv:1912.12180.

Hu, Ronghang, Singh, Amanpreet, Darrell, Trevor, Rohrbach, Marcus, 2020. Iterative answer prediction with pointer-augmented multimodal transformers for textvqa. In: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2020, Seattle, WA, USA, June 13-19, 2020. pp. 9989–9999. http://dx.doi.org/10.1109/CVPR42600.2020.01001.

Huang, Cheng-Zhi Anna, Vaswani, Ashish, Uszkoreit, Jakob, Simon, Ian, Hawthorne, Curtis, Shazeer, Noam, Dai, Andrew M., Hoffman, Matthew D., Dinculescu, Monica, Eck, Douglas, 2019. Music transformer. In: Proceedings of ICLR. URL https://openreview.net/forum?id=rJe4ShAcF7.

Ihm, Hyeong Rae, Lee, Joun Yeop, Choi, Byoung Jin, Cheon, Sung Jun, Kim, Nam Soo, 2020. Reformer-TTS: Neural speech synthesis with reformer network. In: Meng, Helen, Xu, Bo, Zheng, Thomas Fang (Eds.), Proceedings of Interspeech. pp. 2012–2016. http://dx.doi.org/10.21437/Interspeech.2020-2189.

Ioffe, Sergey, Szegedy, Christian, 2015. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In: Proceedings of ICML. pp. 448–456, URL http://proceedings.mlr.press/v37/ioffe15.html.

Irie, Kazuki, Zeyer, Albert, Schlüter, Ralf, Ney, Hermann, 2019. Language modeling with deep transformers. In: Proceedings of Interspeech. pp. 3905–3909. http://dx.doi.org/10.21437/Interspeech.2019-2225.

Islam, Md. Amirul, Jia, Sen, Bruce, Neil D.B., 2020. How much position information do convolutional neural networks encode? In: Proceedings of ICLR. URL https://openreview.net/forum?id=rJeB36NKvB.

Jiang, Yifan, Chang, Shiyu, Wang, Zhangyang, 2021. Transgan: Two transformers can make one strong GAN. arXiv:2102.07074, [cs.CV].

Katharopoulos, Angelos, Vyas, Apoorv, Pappas, Nikolaos, Fleuret, François, 2020. Transformers are RNNs: Fast autoregressive transformers with linear attention. In: Proceedings of ICML. pp. 5156–5165, URL http://proceedings.mlr.press/v119/katharopoulos20a.html.

Ke, Guolin, He, Di, Liu, Tie-Yan, 2020. Rethinking positional encoding in language pre-training. arXiv:2006.15595, [cs.CL].

Khan, Salman, Naseer, Muzammal, Hayat, Munawar, Zamir, Syed Waqas, Khan, Fahad Shahbaz, Shah, Mubarak, 2021. Transformers in vision: A survey. arXiv:2101.01169, [cs.CV].

Kim, Jaeyoung, El-Khamy, Mostafa, Lee, Jungwon, 2020. T-GSA: transformer with Gaussian-weighted self-attention for speech enhancement. In: 2020 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2020, Barcelona, Spain, May 4-8, 2020. IEEE, pp. 6649–6653. http://dx.doi.org/10.1109/ICASSP40776.2020.9053591.

Kitaev, Nikita, Kaiser, Lukasz, Levskaya, Anselm, 2020. Reformer: The efficient transformer. In: Proceedings of ICLR. URL https://openreview.net/forum?id=rkgNKkHtvB.

Klein, Guillaume, Kim, Yoon, Deng, Yuntian, Senellart, Jean, Rush, Alexander, 2017. OpenNMT: Open-source toolkit for neural machine translation. In: Proceedings of ACL. pp. 67–72, URL https://www.aclweb.org/anthology/P17-4012.

Kovaleva, Olga, Romanov, Alexey, Rogers, Anna, Rumshisky, Anna, 2019. Revealing the dark secrets of BERT. In: Proceedings of EMNLP-IJCNLP. pp. 4364–4373. http://dx.doi.org/10.18653/v1/D19-1445.

Lample, Guillaume, Sablayrolles, Alexandre, Ranzato, Marc'Aurelio, Denoyer, Ludovic, Jégou, Hervé, 2019. Large memory layers with product keys. In: Proceedings of NeurIPS. pp. 8546–8557, URL https://proceedings.neurips.cc/paper/2019/hash/9d8df73a3cfbf3c5b47bc9b50f214aff-Abstract.html.

Lee, Juho, Lee, Yoonho, Kim, Jungtaek, Kosiorek, Adam R., Choi, Seungjin, Teh, Yee Whye, 2019. Set transformer: A framework for attention-based permutation-invariant neural networks. In: Proceedings of ICML. pp. 3744–3753, URL http://proceedings.mlr.press/v97/lee19d.html.

Lepikhin, Dmitry, Lee, HyoukJoong, Xu, Yuanzhong, Chen, Dehao, Firat, Orhan, Huang, Yanping, Krikun, Maxim, Shazeer, Noam, Chen, Zhifeng, 2020. Gshard: Scaling giant models with conditional computation and automatic sharding. CoRR arXiv:2006.16668.

Lewis, Mike, Liu, Yinhan, Goyal, Naman, Ghazvininejad, Marjan, Mohamed, Abdelrahman, Levy, Omer, Stoyanov, Veselin, Zettlemoyer, Luke, 2020. BART: denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In: Proceedings of ACL. pp. 7871–7880. http://dx.doi.org/10.18653/v1/2020.acl-main.703.

Li, Wei, Gao, Can, Niu, Guocheng, Xiao, Xinyan, Liu, Hao, Liu, Jiachen, Wu, Hua, Wang, Haifeng, 2020a. UNIMO: Towards unified-modal understanding and generation via cross-modal contrastive learning. arXiv preprint arXiv:2012.15409.

Li, Naihan, Liu, Shujie, Liu, Yanqing, Zhao, Sheng, Liu, Ming, 2019a. Neural speech synthesis with transformer network. In: Proceedings of AAAI. pp. 6706–6713. http://dx.doi.org/10.1609/aaai.v33i01.33016706.

Li, Xiaoya, Meng, Yuxian, Zhou, Mingxin, Han, Qinghong, Wu, Fei, Li, Jiwei, 2020b. SAC: accelerating and structuring self-attention via sparse adaptive connection. In: Proceedings of NeurIPS. URL https://proceedings.neurips.cc/paper/2020/hash/c5c1bda1194f9423d744e0ef67df94ee-Abstract.html.

Li, Xiaonan, Shao, Yunfan, Sun, Tianxiang, Yan, Hang, Qiu, Xipeng, Huang, Xuanjing, 2021. Accelerating BERT inference for sequence labeling via early-exit. arXiv:2105.13878, [cs.CL].

Li, Jian, Tu, Zhaopeng, Yang, Baosong, Lyu, Michael R., Zhang, Tong, 2018. Multi-head attention with disagreement regularization. In: Proceedings of EMNLP. Brussels, Belgium, pp. 2897–2903. http://dx.doi.org/10.18653/v1/D18-1317.

Li, Xiaonan, Yan, Hang, Qiu, Xipeng, Huang, Xuanjing, 2020c. FLAT: Chinese NER using flat-lattice transformer. In: Proceedings of ACL. pp. 6836–6842. http://dx.doi.org/10.18653/v1/2020.acl-main.611.

Li, Jian, Yang, Baosong, Dou, Zi-Yi, Wang, Xing, Lyu, Michael R., Tu, Zhaopeng, 2019b. Information aggregation for multi-head attention with routing-by-agreement. In: Proceedings of HLT-NAACL. pp. 3566–3575. http://dx.doi.org/10.18653/v1/N19-1359.

Li, Liunian Harold, Yatskar, Mark, Yin, Da, Hsieh, Cho-Jui, Chang, Kai-Wei, 2019c. Visualbert: A simple and performant baseline for vision and language. arXiv:1908.03557, [cs.CV].

Lin, Junyang, Men, Rui, Yang, An, Zhou, Chang, Ding, Ming, Zhang, Yichang, Wang, Peng, Wang, Ang, Jiang, Le, Jia, Xianyan, Zhang, Jie, Zhang, Jianwei, Zou, Xu, Li, Zhikang, Deng, Xiaodong, Liu, Jie, Xue, Jinbao, Zhou, Huiling, Ma, Jianxin, Yu, Jin, Li, Yong, Lin, Wei, Zhou, Jingren, Tang, Jie, Yang, Hongxia, 2021. M6: A Chinese multimodal pretrainer. arXiv:2103.00823, [cs.CL].

Liu, Yang, Lapata, Mirella, 2019. Hierarchical transformers for multi-document summarization. In: Proceedings of ACL. Florence, Italy, pp. 5070–5081. http://dx.doi.org/10.18653/v1/P19-1500.

Liu, Ze, Lin, Yutong, Cao, Yue, Hu, Han, Wei, Yixuan, Zhang, Zheng, Lin, Stephen, Guo, Baining, 2021. Swin transformer: Hierarchical vision transformer using shifted windows. arXiv:2103.14030, [cs.CV].

Liu, Liyuan, Liu, Xiaodong, Gao, Jianfeng, Chen, Weizhu, Han, Jiawei, 2020a. Understanding the difficulty of training transformers. In: Proceedings of EMNLP. pp. 5747–5763. http://dx.doi.org/10.18653/v1/2020.emnlp-main.463.

Liu, Yinhan, Ott, Myle, Goyal, Naman, Du, Jingfei, Joshi, Mandar, Chen, Danqi, Levy, Omer, Lewis, Mike, Zettlemoyer, Luke, Stoyanov, Veselin, 2019a. Roberta: A robustly optimized BERT pretraining approach. arXiv:1907.11692, [cs.CL].

Liu, Peter J., Saleh, Mohammad, Pot, Etienne, Goodrich, Ben, Sepassi, Ryan, Kaiser, Lukasz, Shazeer, Noam, 2018. Generating wikipedia by summarizing long sequences. In: Proceedings of ICLR. URL https://openreview.net/forum?id=Hyg0vbWC-.

Liu, Hanxiao, Simonyan, Karen, Yang, Yiming, 2019b. DARTS: Differentiable architecture search. In: Proceedings of ICLR. URL https://openreview.net/forum?id=S1eYHoC5FX.

Liu, Xuanqing, Yu, Hsiang-Fu, Dhillon, Inderjit S., Hsieh, Cho-Jui, 2020b. Learning to encode position for transformer with continuous dynamical model. In: Proceedings of ICML. pp. 6327–6335, URL http://proceedings.mlr.press/v119/liu20n.html.

Lu, Yiping, Li, Zhuohan, He, Di, Sun, Zhiqing, Dong, Bin, Qin, Tao, Wang, Liwei, Liu, Tie-Yan, 2020. Understanding and improving transformer from a multi-particle dynamic system point of view. URL https://openreview.net/forum?id=SJl1o2NFwS.

Ma, Xuezhe, Kong, Xiang, Wang, Sinong, Zhou, Chunting, May, Jonathan, Ma, Hao, Zettlemoyer, Luke, 2021. Luna: Linear unified nested attention. arXiv:2106.01540, [cs.LG].

Mehta, Sachin, Ghazvininejad, Marjan, Iyer, Srinivasan, Zettlemoyer, Luke, Hajishirzi, Hannaneh, 2020. Delight: Very deep and light-weight transformer. arXiv:2008.00623, [cs.LG].

Miculicich, Lesly, Ram, Dhananjay, Pappas, Nikolaos, Henderson, James, 2018. Document-level neural machine translation with hierarchical attention networks. In: Proceedings of EMNLP. Brussels, Belgium, pp. 2947–2954. http://dx.doi.org/10.18653/v1/D18-1325.

Nguyen, Toan Q., Salazar, Julian, 2019. Transformers without tears: Improving the normalization of self-attention. CoRR arXiv:1910.05895.

Parmar, Niki, Vaswani, Ashish, Uszkoreit, Jakob, Kaiser, Lukasz, Shazeer, Noam, Ku, Alexander, Tran, Dustin, 2018. Image transformer. In: Proceedings of ICML. pp. 4052–4061, URL http://proceedings.mlr.press/v80/parmar18a.html.

Peng, Hao, Pappas, Nikolaos, Yogatama, Dani, Schwartz, Roy, Smith, Noah, Kong, Lingpeng, 2021. Random feature attention. In: Proceedings of ICLR. URL https://openreview.net/forum?id=QtTKTdVrFBB.

Perez, Ethan, Strub, Florian, de Vries, Harm, Dumoulin, Vincent, Courville, Aaron C., 2018. Film: Visual reasoning with a general conditioning layer. In: Proceedings of AAAI. pp. 3942–3951, URL https://www.aaai.org/ocs/index.php/AAAI/AAAI18/paper/view/16528.

Pham, Ngoc-Quan, Nguyen, Thai-Son, Niehues, Jan, Müller, Markus, Waibel, Alex, 2019. Very deep self-attention networks for end-to-end speech recognition. In: Proceedings of Interspeech. pp. 66–70. http://dx.doi.org/10.21437/Interspeech.2019-2702.

Pilault, Jonathan, hattami, Amine El, Pal, Christopher, 2021. Conditionally adaptive multi-task learning: Improving transfer learning in NLP using fewer parameters & less data. In: Proceedings of ICLR. URL https://openreview.net/forum?id=de11dbHzAMF.

Press, Ofir, Smith, Noah A., Levy, Omer, 2020. Improving transformer models by reordering their sublayers. In: Proceedings of ACL. pp. 2996–3005. http://dx.doi.org/10.18653/v1/2020.acl-main.270, Online.

Qiu, Xipeng, Sun, TianXiang, Xu, Yige, Shao, Yunfan, Dai, Ning, Huang, Xuanjing, 2020. Pre-trained models for natural language processing: A survey. Sci. China Technol. Sci. 63 (10), 1872–1897. http://dx.doi.org/10.1007/s11431-020-1647-3.

Radford, Alec, Narasimhan, Karthik, Salimans, Tim, Sutskever, Ilya, 2018. Improving language understanding by generative pre-training.

Radford, Alec, Wu, Jeff, Child, Rewon, Luan, David, Amodei, Dario, Sutskever, Ilya, 2019. Language models are unsupervised multitask learners.

Rae, Jack W., Potapenko, Anna, Jayakumar, Siddhant M., Hillier, Chloe, Lillicrap, Timothy P., 2020. Compressive transformers for long-range sequence modelling. In: Proceedings of ICLR. URL https://openreview.net/forum?id=SylKikSYDH.

Raffel, Colin, Shazeer, Noam, Roberts, Adam, Lee, Katherine, Narang, Sharan, Matena, Michael, Zhou, Yanqi, Li, Wei, Liu, Peter J., 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. arXiv:1910.10683, [cs.LG].

Rahimi, Ali, Recht, Benjamin, 2007. Random features for large-scale kernel machines. In: Proceedings of NeurIPS. pp. 1177–1184, URL https://proceedings.neurips.cc/paper/2007/hash/013a006f03dbc5392effeb8f18fda755-Abstract.html.

Ramachandran, Prajit, Zoph, Barret, Le, Quoc V., 2018. Searching for activation functions. In: Proceedings of ICLR. URL https://openreview.net/forum?id=Hkuq2EkPf.

Ramesh, Aditya, Pavlov, Mikhail, Goh, Gabriel, Gray, Scott, Voss, Chelsea, Radford, Alec, Chen, Mark, Sutskever, Ilya, 2021. Zero-shot text-to-image generation. arXiv:2102.12092, [cs.CV].

Rebuffi, Sylvestre-Alvise, Bilen, Hakan, Vedaldi, Andrea, 2017. Learning multiple visual domains with residual adapters. In: Proceedings of NeurIPS. pp. 506–516, URL https://proceedings.neurips.cc/paper/2017/hash/e7b24b112a44fdd9ee93bdf998c6ca0e-Abstract.html.

Rives, Alexander, Meier, Joshua, Sercu, Tom, Goyal, Siddharth, Lin, Zeming, Liu, Jason, Guo, Demi, Ott, Myle, Zitnick, C. Lawrence, Ma, Jerry, Fergus, Rob, 2021. Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences. Proc. Natl. Acad. Sci. 118 (15), http://dx.doi.org/10.1073/pnas.2016239118.

Roller, Stephen, Sukhbaatar, Sainbayar, Szlam, Arthur, Weston, Jason, 2021. Hash layers for large sparse models. arXiv:2106.04426, [cs.LG].

Roy, Aurko, Saffar, Mohammad, Vaswani, Ashish, Grangier, David, 2021. Efficient content-based sparse attention with routing transformers. Transactions of the Association for Computational Linguistics 9, 53–68.

Sabour, Sara, Frosst, Nicholas, Hinton, Geoffrey E., 2017. Dynamic routing between capsules. In: Proceedings of NeurIPS. pp. 3856–3866, URL https://proceedings.neurips.cc/paper/2017/hash/2cad8fa47bbef282badbb8de5374b894-Abstract.html.

Schlag, Imanol, Irie, Kazuki, Schmidhuber, Jürgen, 2021. Linear transformers are secretly fast weight memory systems. CoRR arXiv:2102.11174.

Schwaller, Philippe, Laino, Teodoro, Gaudin, Théophile, Bolgar, Peter, Hunter, Christopher A., Bekas, Costas, Lee, Alpha A., 2019. Molecular transformer: A model for uncertainty-calibrated chemical reaction prediction. ACS Central Sci. 5 (9), 1572–1583. http://dx.doi.org/10.1021/acscentsci.9b00576.

Shao, Jie, Wen, Xin, Zhao, Bingchen, Xue, Xiangyang, 2021. Temporal context aggregation for video retrieval with contrastive learning. In: Proceedings of WACV. pp. 3268–3278.

Shaw, Peter, Uszkoreit, Jakob, Vaswani, Ashish, 2018. Self-attention with relative position representations. In: Proceedings of HLT-NAACL. New Orleans, Louisiana, pp. 464–468. http://dx.doi.org/10.18653/v1/N18-2074.

Shazeer, Noam, 2019. Fast transformer decoding: One write-head is all you need. CoRR arXiv:1911.02150.

Shazeer, Noam, 2020. GLU variants improve transformer. arXiv:2002.05202, [cs.LG].

Shazeer, Noam, Lan, Zhenzhong, Cheng, Youlong, Ding, Nan, Hou, Le, 2020. Talking-heads attention. CoRR arXiv:2003.02436.

Shazeer, Noam, Mirhoseini, Azalia, Maziarz, Krzysztof, Davis, Andy, Le, Quoc V., Hinton, Geoffrey E., Dean, Jeff, 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In: Proceedings of ICLR. URL https://openreview.net/forum?id=B1ckMDqlg.

Shen, Sheng, Yao, Zhewei, Gholami, Amir, Mahoney, Michael W., Keutzer, Kurt, 2020. PowerNorm: Rethinking batch normalization in transformers. In: Proceedings of ICML. pp. 8741–8751, URL http://proceedings.mlr.press/v119/shen20e.html.

Shoeybi, Mohammad, Patwary, Mostofa, Puri, Raul, LeGresley, Patrick, Casper, Jared, Catanzaro, Bryan, 2020. Megatron-LM: Training multi-billion parameter language models using model parallelism. arXiv:1909.08053, [cs.CL].

So, David R., Le, Quoc V., Liang, Chen, 2019. The evolved transformer. In: Proceedings of ICML. pp. 5877–5886, URL http://proceedings.mlr.press/v97/so19a.html.

Su, Jianlin, Lu, Yu, Pan, Shengfeng, Wen, Bo, Liu, Yunfeng, 2021. Roformer: Enhanced transformer with rotary position embedding. arXiv:2104.09864.

Su, Weijie, Zhu, Xizhou, Cao, Yue, Li, Bin, Lu, Lewei, Wei, Furu, Dai, Jifeng, 2020. VL-BERT: pre-training of generic visual-linguistic representations. In: Proceedings of ICLR. URL https://openreview.net/forum?id=SygXPaEYvH.

Sukhbaatar, Sainbayar, Grave, Edouard, Bojanowski, Piotr, Joulin, Armand, 2019a. Adaptive attention span in transformers. In: Proceedings of ACL. Florence, Italy, pp. 331–335. http://dx.doi.org/10.18653/v1/P19-1032.

Sukhbaatar, Sainbayar, Grave, Edouard, Lample, Guillaume, Jegou, Herve, Joulin, Armand, 2019b. Augmenting self-attention with persistent memory. arXiv:1907.01470, [cs.LG].

Sun, Chen, Myers, Austin, Vondrick, Carl, Murphy, Kevin, Schmid, Cordelia, 2019. Videobert: A joint model for video and language representation learning. In: Proceedings of ICCV. pp. 7463–7472. http://dx.doi.org/10.1109/ICCV.2019.00756.

Sun, Tianxiang, Zhou, Yunhua, Liu, Xiangyang, Zhang, Xinyu, Jiang, Hao, Cao, Zhao, Huang, Xuanjing, Qiu, Xipeng, 2021. Early exiting with ensemble internal classifiers. arXiv:2105.13792, [cs.CL].

Sutskever, Ilya, Vinyals, Oriol, Le, Quoc V., 2014. Sequence to sequence learning with neural networks. In: Proceedings of NeurIPS. pp. 3104–3112, URL https://proceedings.neurips.cc/paper/2014/hash/a14ac55a4f27472c5d894ec1c3c743d2-Abstract.html.

Tay, Yi, Bahri, Dara, Metzler, Donald, Juan, Da-Cheng, Zhao, Zhe, Zheng, Che, 2020a. Synthesizer: Rethinking self-attention in transformer models. CoRR arXiv:2005.00743.

Tay, Yi, Bahri, Dara, Yang, Liu, Metzler, Donald, Juan, Da-Cheng, 2020b. Sparse sinkhorn attention. In: Proceedings of ICML. pp. 9438–9447, URL http://proceedings.mlr.press/v119/tay20a.html.

Tsai, Yao-Hung Hubert, Bai, Shaojie, Yamada, Makoto, Morency, Louis-Philippe, Salakhutdinov, Ruslan, 2019. Transformer dissection: An unified understanding for transformer's attention via the lens of kernel. In: Proceedings of EMNLP-IJCNLP. Hong Kong, China, pp. 4344–4353. http://dx.doi.org/10.18653/v1/D19-1443.

van den Oord, Aäron, Dieleman, Sander, Zen, Heiga, Simonyan, Karen, Vinyals, Oriol, Graves, Alex, Kalchbrenner, Nal, Senior, Andrew W., Kavukcuoglu, Koray, 2016. WaveNet: A generative model for raw audio. In: Proceedings of ISCA. p. 125, URL http://www.isca-speech.org/archive/SSW_2016/abstracts/ssw9_DS-4_van_den_Oord.html.

van den Oord, Aäron, Li, Yazhe, Vinyals, Oriol, 2018. Representation learning with contrastive predictive coding. CoRR arXiv:1807.03748.

Vaswani, Ashish, Bengio, Samy, Brevdo, Eugene, Chollet, Francois, Gomez, Aidan, Gouws, Stephan, Jones, Llion, Kaiser, Łukasz, Kalchbrenner, Nal, Parmar, Niki, Sepassi, Ryan, Shazeer, Noam, Uszkoreit, Jakob, 2018. Tensor2Tensor for neural machine translation. In: Proceedings of AMTA. pp. 193–199, URL https://www.aclweb.org/anthology/W18-1819.

Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Lukasz, Polosukhin, Illia, 2017. Attention is all you need. In: Proceedings of NeurIPS. pp. 5998–6008, URL https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

Vyas, Apoorv, Katharopoulos, Angelos, Fleuret, François, 2020. Fast transformers with clustered attention. arXiv:2007.04825, [cs.LG].

Wang, Sinong, Li, Belinda Z., Khabsa, Madian, Fang, Han, Ma, Hao, 2020a. Linformer: Self-attention with linear complexity. arXiv:2006.04768, [cs.LG].

Wang, Qiang, Li, Bei, Xiao, Tong, Zhu, Jingbo, Li, Changliang, Wong, Derek F., Chao, Lidia S., 2019a. Learning deep transformer models for machine translation. In: Proceedings of ACL. pp. 1810–1822. http://dx.doi.org/10.18653/v1/p19-1176.

Wang, Zhiwei, Ma, Yao, Liu, Zitao, Tang, Jiliang, 2019b. R-transformer: Recurrent neural network enhanced transformer. CoRR arXiv:1907.05572.

Wang, Benyou, Shang, Lifeng, Lioma, Christina, Jiang, Xin, Yang, Hao, Liu, Qun, Simonsen, Jakob Grue, 2021a. On position embeddings in BERT. In: Proceedings of ICLR. URL https://openreview.net/forum?id=onxoVA9FxMw.

Wang, Yujing, Yang, Yaming, Bai, Jiangang, Zhang, Mingliang, Bai, Jing, Yu, Jing, Zhang, Ce, Tong, Yunhai, 2021b. Predictive attention transformer: Improving transformer with attention map prediction. URL https://openreview.net/forum?id=YQVjbJPnPc9.

Wang, Benyou, Zhao, Donghao, Lioma, Christina, Li, Qiuchi, Zhang, Peng, Simonsen, Jakob Grue, 2020b. Encoding word order in complex embeddings. In: Proceedings of ICLR. URL https://openreview.net/forum?id=Hke-WTVtwr.

Wu, Felix, Fan, Angela, Baevski, Alexei, Dauphin, Yann N., Auli, Michael, 2019. Pay less attention with lightweight and dynamic convolutions. In: Proceedings of ICLR. URL https://openreview.net/forum?id=SkVhlh09tX.

Wu, Qingyang, Lan, Zhenzhong, Gu, Jing, Yu, Zhou, 2020a. Memformer: The memory-augmented transformer. arXiv:2010.06891, [cs.CL].

Wu, Zhanghao, Liu, Zhijian, Lin, Ji, Lin, Yujun, Han, Song, 2020b. Lite transformer with long-short range attention. In: Proceedings of ICLR. URL https://openreview.net/forum?id=ByeMPlHKPH.

Wu, Zonghan, Pan, Shirui, Chen, Fengwen, Long, Guodong, Zhang, Chengqi, Yu, Philip S., 2021a. A comprehensive survey on graph neural networks. IEEE Trans. Neural Netw. Learn. Syst. 32 (1), 4–24. http://dx.doi.org/10.1109/TNNLS.2020.2978386.

Wu, Chuhan, Wu, Fangzhao, Qi, Tao, Huang, Yongfeng, 2021b. Hi-transformer: Hierarchical interactive transformer for efficient and effective long document modeling. arXiv:2106.01040, [cs.CL].

Xin, Ji, Tang, Raphael, Lee, Jaejun, Yu, Yaoliang, Lin, Jimmy, 2020. DeeBERT: Dynamic early exiting for accelerating BERT inference. In: Proceedings of ACL. pp. 2246–2251. http://dx.doi.org/10.18653/v1/2020.acl-main.204.

Xiong, Ruibin, Yang, Yunchang, He, Di, Zheng, Kai, Zheng, Shuxin, Xing, Chen, Zhang, Huishuai, Lan, Yanyan, Wang, Liwei, Liu, Tie-Yan, 2020. On layer normalization in the transformer architecture. In: Proceedings of ICML. pp. 10524–10533, URL http://proceedings.mlr.press/v119/xiong20b.html.

Xiong, Yunyang, Zeng, Zhanpeng, Chakraborty, Rudrasis, Tan, Mingxing, Fung, Glenn, Li, Yin, Singh, Vikas, 2021. Nyströmformer: A nyström-based algorithm for approximating self-attention. In: Proceedings of AAAI.

Xu, Jingjing, Sun, Xu, Zhang, Zhiyuan, Zhao, Guangxiang, Lin, Junyang, 2019. Understanding and improving layer normalization. In: Proceedings of NeurIPS. pp. 4383–4393, URL https://proceedings.neurips.cc/paper/2019/hash/2f4fe03d77724a7217006e5d16728874-Abstract.html.

Yan, Hang, Deng, Bocao, Li, Xiaonan, Qiu, Xipeng, 2019. TENER: Adapting transformer encoder for named entity recognition. arXiv preprint arXiv:1911.04474.

Yang, An, Lin, Junyang, Men, Rui, Zhou, Chang, Jiang, Le, Jia, Xianyan, Wang, Ang, Zhang, Jie, Wang, Jiamang, Li, Yong, Zhang, Di, Lin, Wei, Qu, Lin, Zhou, Jingren, Yang, Hongxia, 2021. Exploring sparse expert models and beyond. arXiv:2105.15082, [cs.LG].

Yang, Baosong, Tu, Zhaopeng, Wong, Derek F., Meng, Fandong, Chao, Lidia S., Zhang, Tong, 2018. Modeling localness for self-attention networks. In: Proceedings of EMNLP. Brussels, Belgium, pp. 4449–4458. http://dx.doi.org/10.18653/v1/D18-1475.

Yang, Yilin, Wang, Longyue, Shi, Shuming, Tadepalli, Prasad, Lee, Stefan, Tu, Zhaopeng, 2020. On the sub-layer functionalities of transformer decoder. In: Findings of EMNLP. pp. 4799–4811. http://dx.doi.org/10.18653/v1/2020.findings-emnlp.432, Online.

Ye, Zihao, Guo, Qipeng, Gan, Quan, Qiu, Xipeng, Zhang, Zheng, 2019. BP-transformer: Modelling long-range context via binary partitioning. arXiv:1911.04070, [cs.CL].

Ying, Chengxuan, Cai, Tianle, Luo, Shengjie, Zheng, Shuxin, Ke, Guolin, He, Di, Shen, Yanming, Liu, Tie-Yan, 2021a. Do transformers really perform bad for graph representation?. arXiv:2106.05234, [cs.LG].

Ying, Chengxuan, Ke, Guolin, He, Di, Liu, Tie-Yan, 2021b. LazyFormer: Self attention with lazy update. CoRR arXiv:2102.12702.

Yoshida, Davis, Ettinger, Allyson, Gimpel, Kevin, 2020. Adding recurrence to pretrained transformers for improved efficiency and context size. CoRR arXiv:2008.07027.

You, Weiqiu, Sun, Simeng, Iyyer, Mohit, 2020. Hard-coded Gaussian attention for neural machine translation. In: Proceedings of ACL. pp. 7689–7700. http://dx.doi.org/10.18653/v1/2020.acl-main.687, Online.

Yu, Weiwei, Zhou, Jian, Wang, HuaBin, Tao, Liang, 2021. Setransformer: Speech enhancement transformer. Cognit. Comput. http://dx.doi.org/10.1007/s12559-020-09817-2.

Zaheer, Manzil, Guruganesh, Guru, Dubey, Avinava, Ainslie, Joshua, Alberti, Chris, Ontanon, Santiago, Pham, Philip, Ravula, Anirudh, Wang, Qifan, Yang, Li, Ahmed, Amr, 2020. Big bird: Transformers for longer sequences. arXiv:2007.14062, [cs.LG].

Zhang, Hang, Gong, Yeyun, Shen, Yelong, Li, Weisheng, Lv, Jiancheng, Duan, Nan, Chen, Weizhu, 2021. Poolingformer: Long document modeling with pooling attention. arXiv:2105.04371.

Zhang, Xingxing, Wei, Furu, Zhou, Ming, 2019. HIBERT: Document level pre-training of hierarchical bidirectional transformers for document summarization. In: Proceedings of ACL. Florence, Italy, pp. 5059–5069. http://dx.doi.org/10.18653/v1/P19-1499.

Zhang, Biao, Xiong, Deyi, Su, Jinsong, 2018. Accelerating neural transformer via an average attention network. In: Proceedings of ACL. Melbourne, Australia, pp. 1789–1798. http://dx.doi.org/10.18653/v1/P18-1166.

Zhao, Yuekai, Dong, Li, Shen, Yelong, Zhang, Zhihua, Wei, Furu, Chen, Weizhu, 2021. Memory-efficient differentiable transformer architecture search. arXiv:2105.14669, [cs.LG].

Zheng, Minghang, Gao, Peng, Wang, Xiaogang, Li, Hongsheng, Dong, Hao, 2020a. End-to-end object detection with adaptive clustering transformer. CoRR arXiv:2011.09315.

Zheng, Yibin, Li, Xinhui, Xie, Fenglong, Lu, Li, 2020b. Improving end-to-end speech synthesis with local recurrent neural network enhanced transformer. In: Proceedings of ICASSP. pp. 6734–6738. http://dx.doi.org/10.1109/ICASSP40776.2020.9054148.

Zhou, Wangchunshu, Xu, Canwen, Ge, Tao, McAuley, Julian, Xu, Ke, Wei, Furu, 2020. BERT loses patience: Fast and robust inference with early exit. arXiv:2006.04152.

Zhou, Haoyi, Zhang, Shanghang, Peng, Jieqi, Zhang, Shuai, Li, Jianxin, Xiong, Hui, Zhang, Wancai, 2021. Informer: Beyond efficient transformer for long sequence time-series forecasting. In: Proceedings of AAAI.

Zhu, Xizhou, Su, Weijie, Lu, Lewei, Li, Bin, Wang, Xiaogang, Dai, Jifeng, 2020. Deformable DETR: deformable transformers for end-to-end object detection. CoRR arXiv:2010.04159.